

Střední Průmyslová Škola Elektrotechnická Ječná



DLOUHODOBÁ MATURITNÍ PRÁCE

Příloha č.1

Kompletní zdrojový kód

RGB displej 16x16

2026

Radim Žežulka

Obsah

1	Hlavní program a konfigurace.....	3
1.1	Main.ino	3
1.2	Config.h	7
1.3	Buttons.cpp	9
1.4	Buttons.h.....	12
2	Zpracování zvuku	13
2.1	Audio_reactive.cpp	13
2.2	Auduo_reactive.h.....	19
2.3	Vumetr.cpp	19
2.4	Vumetr.h	20
3	Hry a animace.....	21
3.1	Snake.cpp	21
3.2	Snake.h.....	26
3.3	Animations.cpp	26
3.4	Animations.h.....	30
3.5	Sprites.cpp	31
3.6	Sprites.h	50
4	Vzdálené kreslení.....	50
4.1	Drawing.cpp	50
4.2	Drawing.h	53
4.3	Display_utils.cpp	54
4.4	Display_utils.h	57
5	Čas	57
5.1	Clock_wifi.cpp	57
5.2	Clock_wifi.h.....	60

1 Hlavní program a konfigurace

Zdrojové kódy a ostatní části práce v původním formátu jsou dostupné na:

<https://github.com/zezulkaradim/RGB-Display-16-x-16>

1.1 Main.ino

```
#include "config.h"
#include "display_utils.h"
#include "buttons.h"
#include "clock_wifi.h"
#include "animations.h"
#include "snake.h"
#include "vumetr.h"
#include "drawing.h"
#include "sprites.h"
#include "audio_reactive.h"

// ===== Globální proměnné =====

CRGB leds[NUM_LEDS];

DHT dht(DHTPIN, DHTTYPE);
int teplota = 0, vlhkost = 0, teplota1 = 0, vlhkost1 = 0, teplota2 = 0,
vlhkost2 = 0;
uint8_t pocetsekundDHT = 0;

const char* ssid = "Xiaomi 13T Pro";
const char* password = "lupen_smr_di_123";
unsigned long lastTimePrint = 0;
unsigned long lastWiFiCheck = 0;
bool wifiConnecting = false;
unsigned long wifiConnectStart = 0;

const char* ntpServer = "pool.ntp.org";
const long gmtoffset_sec = 2 * 3600;
const int daylightOffset_sec = 0;
int hodina = 0, minuta = 0, hodina1 = 0, hodina2 = 0, minuta1 = 0, minuta2 = 0;
bool dvojtecka = true;
RTC_DS3231 rtc;

int mainmode = 0, nummainmode = 4;
bool setmainmode = true;
int poradianimace = 0, modeanimace = 0, nummodeanimace = 3;
bool setmodeanimace = false, prvnipruchodpacmana = false;
```

```

strukturahada had[256];
bool starthada = false, setmodehad = false;
uint8_t xjablko = 0, yjablko = 0;
bool nahoru = false, dolu = false, doprava = false, doleva = false;
uint8_t delkahada = 0;
int pocetbodu = 0, tisicehad = 0, stovkyhad = 0, desitkyhad = 0, jednotkyhad = 0;
bool setyoulosthad = false, dotekzdi = false, blikyoulost = false;
unsigned long hadpredchozimillis = 0, hadmainlastmillis = 0;
int animacehada = 0;
bool prvnipruchodhada = false;

uint8_t raketaX = 0, raketaY = 0, hueraketa = 0, pozicerakety = HEIGHT;
bool raketaleti = false;
uint8_t fazeexploze = 0;
unsigned long posledniexploze = 0, casFadeStart = 0, posledniRaketa = 0,
posledniPohyb = 0;
bool fadeActive = false;
CRGB ledsohnastroj[NUM_LEDS];
uint8_t heat[WIDTH][HEIGHT];

uint8_t barHeights[16], peak[16], prevFFTValue[16];
bool setmodevumetr = false;

bool setmodekresleni = false;
WebServer server(80);
CRGB ledBuffer[WIDTH][HEIGHT];
bool needsShow = false, kresleniActive = false;

OneButton buttonUp(BUTTON_ARROW_UP, false, false);
OneButton buttonRight(BUTTON_ARROW_RIGHT, false, false);
OneButton buttonDown(BUTTON_ARROW_DOWN, false, false);
OneButton buttonLeft(BUTTON_ARROW_LEFT, false, false);
OneButton buttonHome(BUTTON_HOME, false, false);
OneButton buttonPlay(BUTTON_PLAY, false, false);

// ===== Navigace menu =====

void mainmodes() {
    if (mainmode == 1 && setmainmode) {
        if (!prvnipruchodpacmana) {
            FastLED.clear();
            pacman();
            FastLED.show();
            prvnipruchodpacmana = true;
        }
        EVERY_N_MILLISECONDS(200) {
            FastLED.clear();
            pacman();
        }
    }
}

```

```

        FastLED.show();
    }
} else if (mainmode == 2 && setmainmode) {
    if (!prvnipruchodhada) {
        FastLED.clear();
        mainanimacehada();
        FastLED.show();
        prvnipruchodhada = true;
    }
    EVERY_N_MILLISECONDS(800) {
        FastLED.clear();
        mainanimacehada();
        FastLED.show();
    }
} else if (mainmode == 3 && setmainmode) {
    FastLED.clear();
    animacemikrofon();
    FastLED.show();
} else if (mainmode == 4 && setmainmode) {
    FastLED.clear();
    tuzka();
    FastLED.show();
}
}

// ===== Setup =====

void setup() {
    Serial.begin(115200);
    Wire.begin(21, 22);

    FastLED.addLeds<CHIPSET, DATA_PIN, COLOR_ORDER>(leds, NUM_LEDS);
    FastLED.setMaxPowerInVoltsAndMilliamps(VOLTS, AMPS);
    FastLED.setBrightness(BRIGHTNESS);
    FastLED.clear();
    delay(200);
    FastLED.show();
    delay(200);

    pinMode(BUTTON_ARROW_RIGHT, INPUT_PULLUP);
    pinMode(BUTTON_ARROW_LEFT, INPUT_PULLUP);
    pinMode(BUTTON_ARROW_UP, INPUT_PULLUP);
    pinMode(BUTTON_ARROW_DOWN, INPUT_PULLUP);
    pinMode(BUTTON_HOME, INPUT_PULLUP);
    pinMode(BUTTON_PLAY, INPUT_PULLUP);

    buttonUp.attachClick(clickup);
    buttonRight.attachClick(clickright);
    buttonDown.attachClick(clickdown);

```

```

buttonLeft.attachClick(clickleft);
buttonPlay.attachClick(clickplay);
buttonHome.attachClick(clickhome);

memset(heat, 0, sizeof(heat));

WiFi.mode(WIFI_STA);
WiFi.begin(ssid, password);
wifiConnecting = true;
wifiConnectStart = millis();
lastWiFiCheck = millis();

if (!rtc.begin()) Serial.println("RTC nenalezen.");
if (rtc.lostPower()) Serial.println("RTC bez casu.");

dht.begin();
randomSeed(esp_random());
setupAudio();
}

// ===== Loop =====

void loop() {
    buttonRight.tick();
    buttonLeft.tick();
    buttonPlay.tick();
    buttonHome.tick();
    buttonDown.tick();
    buttonUp.tick();

    if (mainmode == 0 && setmainmode) naticaszwifi();
    else if (modeanimace == 0 && setmodeanimace) rainbowwave();
    else if (modeanimace == 1 && setmodeanimace) rainbowcycle();
    else if (modeanimace == 2 && setmodeanimace) rachejtle();
    else if (modeanimace == 3 && setmodeanimace) Fire2D(50, 90, 120);
    else if (setmodehad && !setmainmode) snake();
    else if (setmodevumetr && !setmainmode) vumetr();
    else if (setmodekresleni && !setmainmode) kresleni();
    else mainmodes();
}

```

1.2 Config.h

```
#pragma once

#include <FastLED.h>
#include <OneButton.h>
#include <WiFi.h>
#include "time.h"
#include "DHT.h"
#include "RTCLib.h"
#include <arduinoFFT.h>
#include <driver/i2s.h>
#include "audio_reactive.h"
#include <WebServer.h>
#include <ArduinoJson.h>

// --- Display ---
#define WIDTH 16
#define HEIGHT 16
#define NUM_LEDS (WIDTH * HEIGHT)
#define DATA_PIN 5
#define COLOR_ORDER GRB
#define CHIPSET WS2812B
#define BRIGHTNESS 200
#define VOLTS 5
#define AMPS 14000
extern CRGB leds[NUM_LEDS];

// --- Tlačítka ---
#define BUTTON_ARROW_UP 25
#define BUTTON_ARROW_RIGHT 26
#define BUTTON_ARROW_DOWN 27
#define BUTTON_ARROW_LEFT 14
#define BUTTON_HOME 33
#define BUTTON_PLAY 13

// --- DHT11 ---
#define DHTPIN 4
#define DHTTYPE DHT11
extern DHT dht;
extern int teplota, vlhkost;
extern int teplota1, vlhkost1;
extern int teplota2, vlhkost2;
extern uint8_t pocetsekundDHT;

// --- Wi-Fi ---
extern const char* ssid;
extern const char* password;
```

```

extern unsigned long lastTimePrint;
extern unsigned long lastWiFiCheck;
extern bool wifiConnecting;
extern unsigned long wifiConnectStart;

// --- Čas / RTC ---
extern const char* ntpServer;
extern const long gmtOffset_sec;
extern const int daylightOffset_sec;
extern int hodina, minuta;
extern int hodina1, hodina2, minuta1, minuta2;
extern bool dvojtecka;
extern RTC_DS3231 rtc;

// --- Navigace / módy ---
extern int mainmode;
extern int nummainmode;
extern bool setmainmode;
extern int poradianimace;
extern int modeanimace;
extern int nummodeanimace;
extern bool setmodeanimace;
extern bool prvnipruchodpacmana;

// --- Had ---
struct strukturahada {
    uint8_t x;
    uint8_t y;
};
extern strukturahada had[256];
extern bool starthada, setmodehad;
extern uint8_t xjablko, yjablko;
extern bool nahoru, dolu, doprava, doleva;
extern uint8_t delkahada;
extern int pocetbodu;
extern int tisicehad, stovkyhad, desitkyhad, jednotkyhad;
extern bool setyoulosthad, dotekzdi, blikyoulost;
extern unsigned long hadpredchozimillis, hadmainlastmillis;
extern int animacehada;
extern bool prvnipruchodhada;

// --- Rachejtle ---
extern uint8_t raketaX, raketaY, hueraketa, pozicerakety;
extern bool raketaleti;
extern uint8_t fazeexploze;
extern unsigned long posledniexploze, casFadeStart, posledniRaketa,
posledniPohyb;
extern bool fadeActive;
extern CRGB ledsohnastroj[NUM_LEDS];

```



```

// --- Oheň ---
extern uint8_t heat[WIDTH][HEIGHT];

// --- VU metr ---
extern uint8_t barHeights[16];
extern uint8_t peak[16];
extern uint8_t prevFFTValue[16];
extern bool setmodevumetr;

// --- Kreslení ---
extern bool setmodekresleni;
extern WebServer server;
extern CRGB ledBuffer[WIDTH][HEIGHT];
extern bool needsShow, kresleniActive;

```

1.3 Buttons.cpp

```

#pragma once
#include "config.h"
#include "display_utils.h"
#include "buttons.h"
#include "clock_wifi.h"
#include "animations.h"
#include "snake.h"
#include "vumetr.h"
#include "drawing.h"
#include "sprites.h"
#include "audio_reactive.h"

void clickright() {
    if (mainmode < nummainmode && setmainmode) mainmode++;
    if (modeanimace < nummodeanimace && setmodeanimace) modeanimace++;
    if (setmodehad && !starthada) {
        starthada = true;
        doprava = true;
    }
    if (setmodehad && starthada && !doleva) {
        dolu = false;
        nahoru = false;
        doprava = true;
        doleva = false;
    }
}

void clickleft() {

```

```

    if (mainmode > 0 && setmainmode) mainmode--;
    if (modeanimace > 0 && setmodeanimace) modeanimace--;
    if (setmodehad && !starthada) {
        starthada = true;
        doleva = true;
    }
    if (setmodehad && starthada && !doprava) {
        dolu = false;
        nahoru = false;
        doprava = false;
        doleva = true;
    }
}

void clickup() {
    if (setmodehad && !starthada) {
        starthada = true;
        nahoru = true;
    }
    if (setmodehad && starthada && !dolu) {
        dolu = false;
        nahoru = true;
        doprava = false;
        doleva = false;
    }
}

void clickdown() {
    if (setmodehad && !starthada) {
        starthada = true;
        dolu = true;
    }
    if (setmodehad && starthada && !nahoru) {
        dolu = true;
        nahoru = false;
        doprava = false;
        doleva = false;
    }
}

void clickplay() {
    if (mainmode == 1 && setmainmode) {
        setmainmode = false;
        setmodeanimace = true;
        modeanimace = 0;
    }
    if (mainmode == 2 && setmainmode) {
        setmainmode = false;
        setmodehad = true;
    }
}

```

```

    starthada = false;
}
if (setyoulosthad && setmodehad) {
    setyoulosthad = false;
    starthada = false;
    xjablko = 0;
    yjablko = 0;
    nahoru = false;
    dolu = false;
    doprava = false;
    doleva = false;
    delkahada = 0;
    pocetbodu = 0;
    tisicehad = 0;
    stovkyhad = 0;
    desitkyhad = 0;
    jednotkyhad = 0;
    dotekzdi = false;
    blikyoulost = false;
    for (int i = 0; i < 255; i++) {
        had[i].x = 0;
        had[i].y = 0;
    }
    hadpredchozimillis = 0;
}
if (mainmode == 3 && setmainmode) {
    setmainmode = false;
    setmodevumetr = true;
}
if (mainmode == 4 && setmainmode) {
    setmainmode = false;
    setmodekresleni = true;
}
}

void clickhome() {
    if (setmodeanimace) {
        setmodeanimace = false;
        setmainmode = true;
        modeanimace = 0;
        mainmode = 1;
    }
    if (setmodehad) {
        setmodehad = false;
        setmainmode = true;
        mainmode = 2;
        setyoulosthad = false;
        starthada = false;
        xjablko = 0;
    }
}

```

```

yjablko = 0;
nahoru = false;
dolu = false;
doprava = false;
doleva = false;
delkahada = 0;
pocetbodu = 0;
tisicehad = 0;
stovkyhad = 0;
desitkyhad = 0;
jednotkyhad = 0;
dotekzdi = false;
blikyoulost = false;
for (int i = 0; i < 255; i++) {
    had[i].x = 0;
    had[i].y = 0;
}
hadpredchozimillis = 0;
}
if (setmodevumetr) {
    setmodevumetr = false;
    setmainmode = true;
    mainmode = 3;
}
if (setmodekresleni) {
    setmodekresleni = false;
    setmainmode = true;
    mainmode = 4;
    stopKresleni();
}
}

```

1.4 Buttons.h

```

#pragma once
#include <Arduino.h>

void clickright();
void clickleft();
void clickup();
void clickdown();
void clickplay();
void clickhome();

```

2 Zpracování zvuku

2.1 Audio_reactive.cpp

```
/*
 * This code has been taken from the sound reactive fork of WLED by Andrew
 * Tuline, and the analog audio processing has been (mostly) removed as we
 * will only be using the INMP441.
 *
 * The FFT code runs on core 0 while everything else runs on core 1. This
 * means we can make our main code more complex without affecting the FFT
 * processing.
 */

#include <driver/i2s.h>
#include "config.h"
#include "display_utils.h"
#include "buttons.h"
#include "clock_wifi.h"
#include "animations.h"
#include "snake.h"
#include "vumetr.h"
#include "drawing.h"
#include "sprites.h"
#include "audio_reactive.h"
#include "arduinoFFT.h"

#define I2S_WS 19    // aka LRCL
#define I2S_SD 23    // aka DOUT
#define I2S_SCK 18   // aka BCLK
#define MIN_SHOW_DELAY 15
const i2s_port_t I2S_PORT = I2S_NUM_0;
const int BLOCK_SIZE = 64;

const int SAMPLE_RATE = 10240;

TaskHandle_t FFT_Task;

int squelch = 0;    // Squelch, cuts out low level sounds
int gain = 30;      // Gain, boosts input level*/
uint16_t micData;   // Analog input for FFT
uint16_t micDataSm; // Smoothed mic data, as it's a bit twitchy

const uint16_t samples = 512; // This value MUST ALWAYS be a power of 2
unsigned int sampling_period_us;
unsigned long microseconds;
```

```

double FFT_MajorPeak = 0;
double FFT_Magnitude = 0;
uint16_t mAvg = 0;

// These are the input and output vectors. Input vectors receive computed
// results from FFT.
double vReal[samples];
double vImag[samples];
double fftBin[samples];

// Try and normalize fftBin values to a max of 4096, so that 4096/16 = 256.
// Oh, and bins 0,1,2 are no good, so we'll zero them out.
double fftCalc[16];
int fftResult[16]; // Our calculated result table, which we feed to the
// animations.
double fftResultMax[16]; // A table used for testing to determine how our
// post-processing is working.

// Table of linearNoise results to be multiplied by squelch in order to reduce
// squelch across fftResult bins.
int linearNoise[16] = { 34, 28, 26, 25, 20, 12, 9, 6, 4, 4, 3, 2, 2, 2, 2, 2 };

// Table of multiplication factors so that we can even out the frequency
// response.
double fftResultPink[16] = { 1.70, 1.71, 1.73, 1.78, 1.68, 1.56, 1.55, 1.63,
1.79, 1.62, 1.80, 2.06, 2.47, 3.35, 6.83, 9.55 };

// Create FFT object
ArduinoFFT<double> FFT(vReal, vImag, samples, (double)SAMPLE_RATE);

double fftAdd(int from, int to) {
    int i = from;
    double result = 0;
    while (i <= to) {
        result += fftBin[i++];
    }
    return result;
}

// FFT main code
void FFTcode(void* parameter) {

    for (;;) {
        delay(1); // DO NOT DELETE THIS LINE! It is needed to give the IDLE(0)
// task enough time and to keep the watchdog happy.
        // taskYIELD(), yield(), vTaskDelay() and esp_task_wdt_feed()
// didn't seem to work.
    }
}

```

```

microseconds = micros();

for (int i = 0; i < samples; i++) {
    int32_t digitalSample = 0;
    size_t bytes_read = 0;
    esp_err_t result = i2s_read(I2S_PORT, &digitalSample,
sizeof(digitalSample), &bytes_read, /*portMAX_DELAY*/ 10);
    //int bytes_read = i2s_pop_sample(I2S_PORT, (char *)&digitalSample,
portMAX_DELAY); // no timeout
    if (bytes_read > 0) micData = abs(digitalSample >> 16);

    vReal[i] = micData; // Store Mic Data in an array
    vImag[i] = 0;
    microseconds += sampling_period_us;
}

FFT.windowing(FFT_WIN_TYP_HAMMING, FFT_FORWARD); // Weigh data
FFT.compute(FFT_FORWARD); // Compute FFT
FFT.complexToMagnitude(); // Compute magnitudes

//
// vReal[3 .. 255] contain useful data, each a 20Hz interval (60Hz -
5120Hz).
// There could be interesting data at bins 0 to 2, but there are too many
artifacts.
//
FFT.majorPeak(&FFT_MajorPeak, &FFT_Magnitude); // let the effects know
which freq was most dominant

for (int i = 0; i < samples; i++) { // Values for bins 0 and 1 are WAY
too large. Might as well start at 3.
    double t = 0.0;
    t = abs(vReal[i]);
    t = t / 16.0; // Reduce magnitude. Want end result to be linear and
~4096 max.
    fftBin[i] = t;
} // for()

/* This FFT post processing is a DIY endeavour. What we really need is
someone with sound engineering expertise to do a great job here AND most
importantly, that the animations look GREAT as a result.
*
* Andrew's updated mapping of 256 bins down to the 16 result bins with
Sample Freq = 10240, samples = 512 and some overlap.
* Based on testing, the lowest/Start frequency is 60 Hz (with bin 3) and
a highest/End frequency of 5120 Hz in bin 255.

```

```

    * Now, Take the 60Hz and multiply by 1.320367784 to get the next
    frequency and so on until the end. Then determine the bins.
    * End frequency = Start frequency * multiplier ^ 16
    * Multiplier = (End frequency/ Start frequency) ^ 1/16
    * Multiplier = 1.320367784
    */

    // Range
    fftCalc[0] = (fftAdd(3, 4)) / 2;           // 60 - 100
    fftCalc[1] = (fftAdd(4, 5)) / 2;           // 80 - 120
    fftCalc[2] = (fftAdd(5, 7)) / 3;           // 100 - 160
    fftCalc[3] = (fftAdd(7, 9)) / 3;           // 140 - 200
    fftCalc[4] = (fftAdd(9, 12)) / 4;           // 180 - 260
    fftCalc[5] = (fftAdd(12, 16)) / 5;          // 240 - 340
    fftCalc[6] = (fftAdd(16, 21)) / 6;          // 320 - 440
    fftCalc[7] = (fftAdd(21, 28)) / 8;          // 420 - 600
    fftCalc[8] = (fftAdd(29, 37)) / 10;         // 580 - 760
    fftCalc[9] = (fftAdd(37, 48)) / 12;         // 740 - 980
    fftCalc[10] = (fftAdd(48, 64)) / 17;        // 960 - 1300
    fftCalc[11] = (fftAdd(64, 84)) / 21;        // 1280 - 1700
    fftCalc[12] = (fftAdd(84, 111)) / 28;       // 1680 - 2240
    fftCalc[13] = (fftAdd(111, 147)) / 37;      // 2220 - 2960
    fftCalc[14] = (fftAdd(147, 194)) / 48;     // 2940 - 3900
    fftCalc[15] = (fftAdd(194, 255)) / 62;     // 3880 - 5120

    // Noise suppression of fftCalc bins using squelch adjustment for different
    input types.
    for (int i = 0; i < 16; i++) {
        fftCalc[i] = fftCalc[i] - (float)squelch * (float)linearNoise[i] / 4.0
    }

    // Adjustment for frequency curves.
    for (int i = 0; i < 16; i++) {
        fftCalc[i] = fftCalc[i] * fftResultPink[i];
    }

    // Manual linear adjustment of gain using gain adjustment for different
    input types.
    for (int i = 0; i < 16; i++) {
        fftCalc[i] = fftCalc[i] * gain / 40 + fftCalc[i] / 16.0;
    }

    // Now, let's dump it all into fftResult. Need to do this, otherwise other
    routines might grab fftResult values prematurely.
    for (int i = 0; i < 16; i++) {
        fftResult[i] = constrain((int)fftCalc[i], 0, 254);
    }

```



```

    } // for(;;)
} // FFTcode()

void setupAudio() {

    // Attempt to configure INMP441 Microphone
    esp_err_t err;
    const i2s_config_t i2s_config = {
        .mode = i2s_mode_t(I2S_MODE_MASTER | I2S_MODE_RX), // Receive, not
transfer
        .sample_rate = SAMPLE_RATE * 2, // 10240 * 2 (20480)
Hz
        .bits_per_sample = I2S_BITS_PER_SAMPLE_32BIT, // could only get it
to work with 32bits
        .channel_format = I2S_CHANNEL_FMT_ONLY_LEFT, // LEFT when pin is
tied to ground.
        .communication_format = i2s_comm_format_t(I2S_COMM_FORMAT_I2S |
I2S_COMM_FORMAT_I2S_MSB),
        .intr_alloc_flags = ESP_INTR_FLAG_LEVEL1, // Interrupt level 1
        .dma_buf_count = 8, // number of buffers
        .dma_buf_len = BLOCK_SIZE // samples per buffer
    };
    const i2s_pin_config_t pin_config = {
        .bck_io_num = I2S_SCK, // BCLK aka SCK
        .ws_io_num = I2S_WS, // LRCLK aka WS
        .data_out_num = -1, // not used (only for speakers)
        .data_in_num = I2S_SD // DOUT aka SD
    };

    // Configuring the I2S driver and pins.
    // This function must be called before any I2S driver read/write operations.
    err = i2s_driver_install(I2S_PORT, &i2s_config, 0, NULL);
    if (err != ESP_OK) {
        Serial.printf("Failed installing driver: %d\n", err);
        while (true)
            ;
    }
    err = i2s_set_pin(I2S_PORT, &pin_config);
    if (err != ESP_OK) {
        Serial.printf("Failed setting pin: %d\n", err);
        while (true)
            ;
    }

    // Test to see if we have a digital microphone installed or not.
    float mean = 0.0;

```

```

int32_t samples[BLOCK_SIZE];
size_t num_bytes_read = 0;

esp_err_t result = i2s_read(I2S_PORT, &samples, BLOCK_SIZE, &num_bytes_read,
portMAX_DELAY);

/*int num_bytes_read = i2s_read_bytes(I2S_PORT,
                                     (char *)samples,
                                     BLOCK_SIZE,      // the doc says bytes,
but its elements.
                                     portMAX_DELAY); // no timeout*/

int samples_read = num_bytes_read / 8;
if (samples_read > 0) {
    for (int i = 0; i < samples_read; ++i) {
        mean += samples[i];
    }
    mean = mean / BLOCK_SIZE / 16384;
    if (mean != 0.0) {
    } else {
        Serial.println("Digital microphone is NOT present.");
    }
}

sampling_period_us = round(1000000 * (1.0 / SAMPLE_RATE));

// Define the FFT Task and lock it to core 0
xTaskCreatePinnedToCore(
    FFTcode,      // Function to implement the task
    "FFT",        // Name of the task
    10000,        // Stack size in words
    NULL,         // Task input parameter
    1,            // Priority of the task
    &FFT_Task,    // Task handle
    0);           // Core where the task should run
} //setupAudio()

```

2.2 Audio_reactive.h

```
#pragma once
#include <Arduino.h>

extern int fftResult[16];
extern int gain;
extern int squelch;

void setupAudio();
void FFTcode(void* parameter);
double fftAdd(int from, int to);
```

2.3 Vumetr.cpp

```
#pragma once

#include "config.h"
#include "display_utils.h"
#include "buttons.h"
#include "clock_wifi.h"
#include "animations.h"
#include "snake.h"
#include "vumetr.h"
#include "drawing.h"
#include "sprites.h"
#include "audio_reactive.h" // Kvůli funkci XY()

void drawRainbowBar(uint8_t band, uint8_t height) {
    for (uint8_t y = 0; y <= height; y++) {
        leds[XY(band, (HEIGHT - 1) - y)] = CHSV(map(band, 0, 15, 0, 255), 255, 255);
    }
}

void drawWhitePeak(uint8_t band) {
    leds[XY(band, (HEIGHT - 1) - peak[band])] = CRGB::White;
}

void decayPeaks() {
    static uint32_t lastDecay = 0;
    if (millis() - lastDecay > 60) {
        lastDecay = millis();
        for (uint8_t i = 0; i < 16; i++) {
            if (peak[i] > 0) peak[i]--;
        }
    }
}
```

```

void vumetr() {
    FastLED.clear();
    for (uint8_t band = 0; band < 16; band++) {
        uint8_t fftVal = ((prevFFTValue[band] * 3) + fftResult[band]) / 4;
        prevFFTValue[band] = fftVal;
        uint8_t height = map(fftVal, 0, 255, 0, HEIGHT - 1);
        barHeights[band] = height;

        if (height > peak[band]) peak[band] = height;

        drawRainbowBar(band, height);
        drawWhitePeak(band);
    }
    decayPeaks();
    FastLED.show();
}

```

2.4 Vumetr.h

```

#pragma once
#include <Arduino.h>

void drawRainbowBar(uint8_t band, uint8_t height);
void drawWhitePeak(uint8_t band);
void decayPeaks();
void vumetr();

```

3 Hry a animace

3.1 Snake.cpp

```
#pragma once
#include "config.h"
#include "display_utils.h"
#include "buttons.h"
#include "clock_wifi.h"
#include "animations.h"
#include "snake.h"
#include "vumetr.h"
#include "drawing.h"
#include "sprites.h"
#include "audio_reactive.h"
void youlosthad() {
    starthada = false;
    setyoulosthad = true;
    unsigned long hadmillisted = millis();
    if (hadmillisted - hadpredchozimillis >= 400) {
        hadpredchozimillis = hadmillisted;
        blikyoulost = !blikyoulost;
    }

    FastLED.clear();
    if (blikyoulost) {
        // YOU LOST text pixel-art
        int x = 0;
        for (int i = 0; i <= 9; i++)
            if (i < 2 || i > 4) leds[XY(x, i)] = CRGB(255, 0, 0);
        x = 1;
        for (int i = 0; i <= 9; i++)
            if ((i > 0 && i < 4) || i > 8) leds[XY(x, i)] = CRGB(255, 0, 0);
        x = 2;
        for (int i = 0; i <= 9; i++)
            if (i < 2 || i > 8) leds[XY(x, i)] = CRGB(255, 0, 0);
        x = 4;
        for (int i = 0; i <= 9; i++)
            if (i < 4 || i > 4) leds[XY(x, i)] = CRGB(255, 0, 0);
        x = 5;
        leds[XY(x, 0)] = CRGB(255, 0, 0);
        leds[XY(x, 3)] = CRGB(255, 0, 0);
        leds[XY(x, 5)] = CRGB(255, 0, 0);
        leds[XY(x, 9)] = CRGB(255, 0, 0);
        x = 6;
        for (int i = 0; i <= 9; i++)
            if (i < 4 || i > 4) leds[XY(x, i)] = CRGB(255, 0, 0);
```

```

x = 8;
for (int i = 0; i <= 7; i++)
    if (i < 4 || i > 4) leds[XY(x, i)] = CRGB(255, 0, 0);
leds[XY(x, 9)] = CRGB(255, 0, 0);
x = 9;
leds[XY(x, 9)] = CRGB(255, 0, 0);
leds[XY(x, 7)] = CRGB(255, 0, 0);
leds[XY(x, 5)] = CRGB(255, 0, 0);
leds[XY(x, 3)] = CRGB(255, 0, 0);
x = 10;
for (int i = 0; i <= 9; i++)
    if (i < 4 || i > 6) leds[XY(x, i)] = CRGB(255, 0, 0);
leds[XY(x, 5)] = CRGB(255, 0, 0);
x = 12;
leds[XY(x, 5)] = CRGB(255, 0, 0);
x = 13;
for (int i = 5; i <= 9; i++) leds[XY(x, i)] = CRGB(255, 0, 0);
x = 14;
leds[XY(x, 5)] = CRGB(255, 0, 0);
FastLED.show();
blikyoulost = true;
} else {
    for (int i = 0; i <= 159; i++) leds[i] = CRGB::Black;
}

tisicehad = pocetbodu / 1000;
stovkyhad = (pocetbodu / 100) % 10;
desitkyhad = (pocetbodu / 10) % 10;
jednotkyhad = pocetbodu % 10;
vykreslicislo(0, 15, tisicehad, 255, 255, 255);
vykreslicislo(4, 15, stovkyhad, 255, 255, 255);
vykreslicislo(8, 15, desitkyhad, 255, 255, 255);
vykreslicislo(12, 15, jednotkyhad, 255, 255, 255);
FastLED.show();
}

void generujjablko() {
    bool chyba;
    do {
        chyba = false;
        xjablko = random(1, 15);
        yjablko = random(1, 15);
        for (int i = 0; i <= delkahada; i++)
            if (xjablko == had[i].x && yjablko == had[i].y) {
                chyba = true;
                break;
            }
    } while (chyba);
}

```

```

void haddoprava() {
    if (had[0].x == xjablko && had[0].y == yjablko) {
        delkahada++;
        pocetbodu++;
        for (int i = delkahada; i > 0; i--) {
            had[i].x = had[i - 1].x;
            had[i].y = had[i - 1].y;
        }
        had[0].x++;
        generujjablko();
        return;
    }
    for (int i = 0; i <= delkahada; i++)
        if (had[0].x + 1 == had[i].x && had[0].y == had[i].y) {
            youlosthad();
            return;
        }
    if (had[0].x + 1 > 15) {
        if (dotekzdi) youlosthad();
        else dotekzdi = true;
        return;
    }
    dotekzdi = false;
    for (int i = delkahada; i > 0; i--) {
        had[i].x = had[i - 1].x;
        had[i].y = had[i - 1].y;
    }
    had[0].x++;
}

void haddoleva() {
    if (had[0].x == xjablko && had[0].y == yjablko) {
        delkahada++;
        pocetbodu++;
        generujjablko();
        for (int i = delkahada; i > 0; i--) {
            had[i].x = had[i - 1].x;
            had[i].y = had[i - 1].y;
        }
        had[0].x--;
        return;
    }
    for (int i = 0; i <= delkahada; i++)
        if (had[0].x - 1 == had[i].x && had[0].y == had[i].y) {
            youlosthad();
            return;
        }
    if (had[0].x - 1 < 0) {

```

```

        if (dotekzdi) youlosthad();
        else dotekzdi = true;
        return;
    }
    dotekzdi = false;
    for (int i = delkahada; i > 0; i--) {
        had[i].x = had[i - 1].x;
        had[i].y = had[i - 1].y;
    }
    had[0].x--;
}

void hadnahoru() {
    if (had[0].x == xjablko && had[0].y == yjablko) {
        delkahada++;
        pocetbodu++;
        generujjablko();
        for (int i = delkahada; i > 0; i--) {
            had[i].x = had[i - 1].x;
            had[i].y = had[i - 1].y;
        }
        had[0].y--;
        return;
    }
    for (int i = 0; i <= delkahada; i++)
        if (had[0].x == had[i].x && had[0].y - 1 == had[i].y) {
            youlosthad();
            return;
        }
    if (had[0].y - 1 < 0) {
        if (dotekzdi) youlosthad();
        else dotekzdi = true;
        return;
    }
    dotekzdi = false;
    for (int i = delkahada; i > 0; i--) {
        had[i].x = had[i - 1].x;
        had[i].y = had[i - 1].y;
    }
    had[0].y--;
}

void haddolu() {
    if (had[0].x == xjablko && had[0].y == yjablko) {
        delkahada++;
        pocetbodu++;
        generujjablko();
        for (int i = delkahada; i > 0; i--) {
            had[i].x = had[i - 1].x;

```



```

        had[i].y = had[i - 1].y;
    }
    had[0].y++;
    return;
}
for (int i = 0; i <= delkahada; i++)
    if (had[0].x == had[i].x && had[0].y + 1 == had[i].y) {
        youlosthad();
        return;
    }
if (had[0].y + 1 > 15) {
    if (dotekzdi) youlosthad();
    else dotekzdi = true;
    return;
}
dotekzdi = false;
for (int i = delkahada; i > 0; i--) {
    had[i].x = had[i - 1].x;
    had[i].y = had[i - 1].y;
}
had[0].y++;
}

void snake() {
    if (setyoulosthad) {
        youlosthad();
        return;
    }

    if (!starthada) {
        FastLED.clear();
        while (xjablko == had[0].x && yjablko == had[0].y) {
            xjablko = random(1, 15);
            yjablko = random(1, 15);
            had[0].x = random(3, 13);
            had[0].y = random(3, 13);
        }
        leds[XY(xjablko, yjablko)] = CRGB(255, 0, 0);
        leds[XY(had[0].x, had[0].y)] = CRGB(0, 255, 0);
        FastLED.show();
    }

    unsigned long now = millis();
    if (now - hadmainlastmillis < (unsigned long)(500 - (5 * pocetbodu)))
        return;
    hadmainlastmillis = now;

    if (starthada) {
        if (doprava) haddoprava();
    }
}

```

```

    if (doleva) haddoleva();
    if (dolu) haddolu();
    if (nahoru) hadnahoru();
    FastLED.clear();
    leds[XY(xjablko, yjablko)] = CRGB(255, 0, 0);
    for (int i = 0; i <= delkahada; i++)
        leds[XY(had[i].x, had[i].y)] = (i % 2 == 0) ? CRGB(0, 125, 0) : CRGB(0,
255, 0);
    FastLED.show();
}
}

```

3.2 Snake.h

```

#pragma once
#include <Arduino.h>

```

```

void youlosthad();
void generujjablko();
void haddoprava();
void haddoleva();
void hadnahoru();
void haddolu();
void snake();

```

3.3 Animations.cpp

```

#pragma once
#include "config.h"
#include "display_utils.h"
#include "buttons.h"
#include "clock_wifi.h"
#include "animations.h"
#include "snake.h"
#include "vumetr.h"
#include "drawing.h"
#include "sprites.h"
#include "audio_reactive.h"

void rainbowwave() {
    FastLED.clear();
    static uint16_t t = 0;
    EVERY_N_MILLISECONDS(5) {
        t++;
        for (uint8_t y = 0; y < HEIGHT; y++)
            for (uint8_t x = 0; x < WIDTH; x++)
                leds[XY(x, y)] = CHSV((x * 6 + y * 6 + t) % 255, 255, 255);
        FastLED.show();
    }
}

```

```

}

void rainbowcycle() {
    FastLED.clear();
    static uint16_t t = 0;
    EVERY_N_MILLISECONDS(5) {
        t++;
        float cx = (WIDTH - 1) / 2.0, cy = (HEIGHT - 1) / 2.0;
        for (uint8_t y = 0; y < HEIGHT; y++) {
            for (uint8_t x = 0; x < WIDTH; x++) {
                float dx = x - cx, dy = y - cy;
                uint8_t hue = (uint8_t)(atan2(dy, dx) * 40 + sqrt(dx * dx + dy * dy) *
15 + t);
                leds[XY(x, y)] = CHSV(hue, 255, 255);
            }
        }
        FastLED.show();
    }
}

// --- Rachejtle ---

void exploze1() {
    leds[XY(raketaX, raketaY)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX + 1, raketaY)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX + 1, raketaY + 1)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX, raketaY + 1)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX - 1, raketaY + 1)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX - 1, raketaY)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX - 1, raketaY - 1)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX, raketaY - 1)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX + 1, raketaY - 1)] = CHSV(hueraketa, 255, 255);
}

void exploze2() {
    exploze1();
    leds[XY(raketaX + 2, raketaY)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX + 3, raketaY)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX, raketaY + 2)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX, raketaY + 3)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX - 2, raketaY)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX - 3, raketaY)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX, raketaY - 2)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX, raketaY - 3)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX + 2, raketaY + 1)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX + 1, raketaY + 2)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX - 1, raketaY + 2)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX - 2, raketaY + 1)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX - 1, raketaY - 2)] = CHSV(hueraketa, 255, 255);
}

```

```

    leds[XY(raketaX + 1, raketaY - 2)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX + 2, raketaY - 1)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX - 2, raketaY - 1)] = CHSV(hueraketa, 255, 255);
}

void exploze3() {
    exploze2();
    leds[XY(raketaX + 3, raketaY + 1)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX + 3, raketaY + 2)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX + 3, raketaY - 1)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX + 3, raketaY - 2)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX + 2, raketaY + 3)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX + 1, raketaY + 3)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX - 1, raketaY + 3)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX - 2, raketaY + 3)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX - 3, raketaY + 2)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX - 3, raketaY + 1)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX - 3, raketaY - 1)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX - 3, raketaY - 2)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX - 2, raketaY - 3)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX - 1, raketaY - 3)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX + 1, raketaY - 3)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX + 2, raketaY - 3)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX + 2, raketaY - 2)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX + 2, raketaY + 2)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX - 2, raketaY + 2)] = CHSV(hueraketa, 255, 255);
    leds[XY(raketaX - 2, raketaY - 2)] = CHSV(hueraketa, 255, 255);
}

void rachejtle() {
    unsigned long now = millis();

    if (!raketaleti && !fadeActive && (now - posledniRaketa >= 1500)) {
        raketaX = random(3, 13);
        raketaY = random(3, 13);
        pozicerakety = HEIGHT - 1;
        hueraketa += 15;
        raketaleti = true;
        fazeexploze = 0;
        posledniRaketa = now;
    }

    if (raketaleti && now - posledniPohyb > 20) {
        FastLED.clear();
        leds[XY(raketaX, pozicerakety)] = CRGB::White;
        FastLED.show();
        pozicerakety--;
        posledniPohyb = now;
        if (pozicerakety <= raketaY) {

```

```

        raketaleti = false;
        fazeexploze = 1;
        posledniexploze = now;
    }
}

if (fazeexploze >= 1 && fazeexploze <= 3 && !fadeActive) {
    if (now - posledniexploze >= 80) {
        FastLED.clear();
        if (fazeexploze == 1) exploze1();
        else if (fazeexploze == 2) exploze2();
        else if (fazeexploze == 3) {
            exploze3();
            for (int i = 0; i < NUM_LEDS; i++) ledsohnastroj[i] = leds[i];
            fadeActive = true;
            casFadeStart = now;
        }
        FastLED.show();
        fazeexploze++;
        posledniexploze = now;
    }
}

if (fadeActive) {
    unsigned long elapsed = now - casFadeStart;
    if (elapsed <= 1000) {
        uint8_t brightness = map(elapsed, 0, 1000, 255, 0);
        for (int i = 0; i < NUM_LEDS; i++) {
            leds[i] = ledsohnastroj[i];
            leds[i].nscale8_video(brightness);
        }
        FastLED.show();
    } else {
        fadeActive = false;
        FastLED.clear();
        FastLED.show();
    }
}
}

// --- Oheň ---

void Fire2D(uint8_t cooling, uint8_t sparking, uint8_t speedDelay) {
    static unsigned long last = 0;
    unsigned long now = millis();
    if (now - last < speedDelay) return;
    last = now;

    for (uint8_t x = 0; x < WIDTH; x++)

```

```

    for (uint8_t y = 0; y < HEIGHT; y++)
        heat[x][y] = qsub8(heat[x][y], random8(0, (cooling * 10 / HEIGHT) + 2));

uint8_t newHeat[WIDTH][HEIGHT];
memset(newHeat, 0, sizeof(newHeat));

for (uint8_t x = 0; x < WIDTH; x++) {
    for (uint8_t y = 0; y < HEIGHT; y++) {
        uint16_t sum = heat[x][y];
        uint8_t count = 1;
        uint8_t yb = (y < HEIGHT - 1) ? y + 1 : y;
        uint8_t xl = (x == 0) ? WIDTH - 1 : x - 1;
        uint8_t xr = (x == WIDTH - 1) ? 0 : x + 1;
        sum += heat[x][yb];
        count++;
        sum += heat[xl][yb];
        count++;
        sum += heat[xr][yb];
        count++;
        newHeat[x][y] = sum / count;
    }
}
memcpy(heat, newHeat, sizeof(heat));

for (uint8_t x = 0; x < WIDTH; x++)
    if (random8() < sparking)
        heat[x][HEIGHT - 1] = qadd8(heat[x][HEIGHT - 1], random8(160, 255));

for (uint8_t x = 0; x < WIDTH; x++)
    for (uint8_t y = 0; y < HEIGHT; y++)
        leds[XY(x, y)] = ColorFromPalette(HeatColors_p, scale8(heat[x][y],
240));

FastLED.show();
}

```

3.4 Animations.h

```

#pragma once
#include <Arduino.h>

void rainbowwave();
void rainbowcycle();
void exploze1();
void exploze2();
void exploze3();
void rachejtle();
void Fire2D(uint8_t cooling=55, uint8_t sparking =120, uint8_t speedDelay=50);

```

3.5 Sprites.cpp

```
#pragma once
#include "config.h"
#include "display_utils.h"
#include "buttons.h"
#include "clock_wifi.h"
#include "animations.h"
#include "snake.h"
#include "vumetr.h"
#include "drawing.h"
#include "sprites.h"
#include "audio_reactive.h"

// ===== HAD - animace na menu =====

void telohada1() {
    int x = 0;
    for (int i = 0; i <= 15; i++)
        if (i > 10 && i < 14) leds[XY(x, i)] = CRGB(0, 80, 0);
    x = 1;
    for (int i = 0; i <= 15; i++)
        if (i > 7 && i < 12) leds[XY(x, i)] = CRGB(0, 80, 0);
    leds[XY(1, 14)] = CRGB(0, 80, 0);
    leds[XY(1, 15)] = CRGB(0, 80, 0);
    x = 2;
    leds[XY(x, 10)] = CRGB(0, 80, 0);
    leds[XY(x, 11)] = CRGB(0, 80, 0);
    leds[XY(x, 15)] = CRGB(0, 80, 0);
    x = 3;
    leds[XY(x, 10)] = CRGB(0, 80, 0);
    leds[XY(x, 11)] = CRGB(0, 80, 0);
    leds[XY(x, 15)] = CRGB(0, 80, 0);
    x = 4;
    leds[XY(x, 11)] = CRGB(0, 80, 0);
    leds[XY(x, 15)] = CRGB(0, 80, 0);
    x = 5;
    leds[XY(x, 11)] = CRGB(0, 80, 0);
    leds[XY(x, 15)] = CRGB(0, 80, 0);
    x = 6;
    leds[XY(x, 8)] = CRGB(0, 80, 0);
    leds[XY(x, 11)] = CRGB(0, 80, 0);
    leds[XY(x, 12)] = CRGB(0, 80, 0);
    leds[XY(x, 15)] = CRGB(0, 80, 0);
    for (x = 7; x < 13; x++) {
        leds[XY(x, 8)] = CRGB(0, 80, 0);
        leds[XY(x, 12)] = CRGB(0, 80, 0);
        leds[XY(x, 15)] = CRGB(0, 80, 0);
    }
}
```

```

}
x = 13;
leds[XY(x, 8)] = CRGB(0, 80, 0);
leds[XY(x, 9)] = CRGB(0, 80, 0);
leds[XY(x, 12)] = CRGB(0, 80, 0);
leds[XY(x, 13)] = CRGB(0, 80, 0);
leds[XY(x, 15)] = CRGB(0, 80, 0);
x = 14;
for (int i = 0; i <= 15; i++)
    if (i > 10) leds[XY(x, i)] = CRGB(0, 80, 0);
leds[XY(14, 9)] = CRGB(0, 80, 0);
x = 15;
for (int i = 0; i <= 15; i++)
    if ((i > 8 && i < 15) && (i != 12)) leds[XY(x, i)] = CRGB(0, 80, 0);

x = 2;
leds[XY(x, 13)] = CRGB(70, 255, 0);
x = 3;
leds[XY(x, 8)] = CRGB(70, 255, 0);
leds[XY(x, 9)] = CRGB(70, 255, 0);
leds[XY(x, 13)] = CRGB(70, 255, 0);
leds[XY(x, 14)] = CRGB(70, 255, 0);
x = 4;
leds[XY(x, 8)] = CRGB(70, 255, 0);
leds[XY(x, 9)] = CRGB(70, 255, 0);
leds[XY(x, 10)] = CRGB(70, 255, 0);
leds[XY(x, 13)] = CRGB(70, 255, 0);
leds[XY(x, 14)] = CRGB(70, 255, 0);
x = 5;
leds[XY(x, 9)] = CRGB(70, 255, 0);
leds[XY(x, 10)] = CRGB(70, 255, 0);
leds[XY(x, 13)] = CRGB(70, 255, 0);
leds[XY(x, 14)] = CRGB(70, 255, 0);
x = 6;
leds[XY(x, 9)] = CRGB(70, 255, 0);
leds[XY(x, 10)] = CRGB(70, 255, 0);
leds[XY(x, 13)] = CRGB(70, 255, 0);
leds[XY(x, 14)] = CRGB(70, 255, 0);
x = 7;
leds[XY(x, 9)] = CRGB(70, 255, 0);
leds[XY(x, 10)] = CRGB(70, 255, 0);
leds[XY(x, 14)] = CRGB(70, 255, 0);
for (x = 7; x < 12; x++) {
    leds[XY(x, 9)] = CRGB(70, 255, 0);
    leds[XY(x, 10)] = CRGB(70, 255, 0);
    leds[XY(x, 11)] = CRGB(70, 255, 0);
    leds[XY(x, 14)] = CRGB(70, 255, 0);
}
x = 12;

```



```

    leds[XY(x, 10)] = CRGB(70, 255, 0);
    leds[XY(x, 11)] = CRGB(70, 255, 0);
    leds[XY(x, 14)] = CRGB(70, 255, 0);
    x = 13;
    leds[XY(x, 10)] = CRGB(70, 255, 0);

    x = 1;
    leds[XY(x, 12)] = CRGB(0, 255, 0);
    leds[XY(x, 13)] = CRGB(0, 255, 0);
    x = 2;
    leds[XY(x, 8)] = CRGB(0, 255, 0);
    leds[XY(x, 9)] = CRGB(0, 255, 0);
    leds[XY(x, 12)] = CRGB(0, 255, 0);
    leds[XY(x, 14)] = CRGB(0, 255, 0);
    x = 3;
    leds[XY(x, 12)] = CRGB(0, 255, 0);
    x = 4;
    leds[XY(x, 12)] = CRGB(0, 255, 0);
    x = 5;
    leds[XY(x, 8)] = CRGB(0, 255, 0);
    leds[XY(x, 12)] = CRGB(0, 255, 0);
    x = 7;
    leds[XY(x, 11)] = CRGB(0, 255, 0);
    leds[XY(x, 13)] = CRGB(0, 255, 0);
    for (x = 7; x < 12; x++) leds[XY(x, 13)] = CRGB(70, 255, 0);
    x = 12;
    leds[XY(x, 9)] = CRGB(0, 255, 0);
    leds[XY(x, 13)] = CRGB(0, 255, 0);
    x = 13;
    leds[XY(x, 11)] = CRGB(0, 255, 0);
    leds[XY(x, 14)] = CRGB(0, 255, 0);
    x = 14;
    leds[XY(x, 10)] = CRGB(0, 255, 0);
}

void telohada2() {
    int x = 0;
    for (int i = 0; i <= 15; i++)
        if (i > 10 && i < 14) leds[XY(x, i)] = CRGB(0, 80, 0);
    x = 1;
    for (int i = 0; i <= 15; i++)
        if (i > 7 && i < 12) leds[XY(x, i)] = CRGB(0, 80, 0);
    leds[XY(1, 14)] = CRGB(0, 80, 0);
    leds[XY(1, 15)] = CRGB(0, 80, 0);
    x = 2;
    leds[XY(x, 8)] = CRGB(0, 80, 0);
    leds[XY(x, 10)] = CRGB(0, 80, 0);
    leds[XY(x, 11)] = CRGB(0, 80, 0);
    leds[XY(x, 15)] = CRGB(0, 80, 0);
}

```

```

x = 3;
leds[XY(x, 10)] = CRGB(0, 80, 0);
leds[XY(x, 11)] = CRGB(0, 80, 0);
leds[XY(x, 15)] = CRGB(0, 80, 0);
x = 4;
leds[XY(x, 11)] = CRGB(0, 80, 0);
leds[XY(x, 15)] = CRGB(0, 80, 0);
x = 5;
leds[XY(x, 11)] = CRGB(0, 80, 0);
leds[XY(x, 15)] = CRGB(0, 80, 0);
x = 6;
leds[XY(x, 8)] = CRGB(0, 80, 0);
leds[XY(x, 11)] = CRGB(0, 80, 0);
leds[XY(x, 12)] = CRGB(0, 80, 0);
leds[XY(x, 15)] = CRGB(0, 80, 0);
for (x = 7; x < 13; x++) {
    leds[XY(x, 8)] = CRGB(0, 80, 0);
    leds[XY(x, 12)] = CRGB(0, 80, 0);
    leds[XY(x, 15)] = CRGB(0, 80, 0);
}
x = 13;
leds[XY(x, 8)] = CRGB(0, 80, 0);
leds[XY(x, 9)] = CRGB(0, 80, 0);
leds[XY(x, 12)] = CRGB(0, 80, 0);
leds[XY(x, 13)] = CRGB(0, 80, 0);
leds[XY(x, 15)] = CRGB(0, 80, 0);
x = 14;
for (int i = 0; i <= 15; i++)
    if (i > 10) leds[XY(x, i)] = CRGB(0, 80, 0);
leds[XY(14, 9)] = CRGB(0, 80, 0);
x = 15;
for (int i = 0; i <= 15; i++)
    if ((i > 8 && i < 15) && (i != 12)) leds[XY(x, i)] = CRGB(0, 80, 0);

x = 2;
leds[XY(x, 13)] = CRGB(70, 255, 0);
x = 3;
leds[XY(x, 8)] = CRGB(70, 255, 0);
leds[XY(x, 9)] = CRGB(70, 255, 0);
leds[XY(x, 13)] = CRGB(70, 255, 0);
leds[XY(x, 14)] = CRGB(70, 255, 0);
x = 4;
leds[XY(x, 8)] = CRGB(70, 255, 0);
leds[XY(x, 9)] = CRGB(70, 255, 0);
leds[XY(x, 10)] = CRGB(70, 255, 0);
leds[XY(x, 13)] = CRGB(70, 255, 0);
leds[XY(x, 14)] = CRGB(70, 255, 0);
x = 5;
leds[XY(x, 9)] = CRGB(70, 255, 0);

```

```

leds[XY(x, 10)] = CRGB(70, 255, 0);
leds[XY(x, 13)] = CRGB(70, 255, 0);
leds[XY(x, 14)] = CRGB(70, 255, 0);
x = 6;
leds[XY(x, 9)] = CRGB(70, 255, 0);
leds[XY(x, 10)] = CRGB(70, 255, 0);
leds[XY(x, 13)] = CRGB(70, 255, 0);
leds[XY(x, 14)] = CRGB(70, 255, 0);
x = 7;
leds[XY(x, 9)] = CRGB(70, 255, 0);
leds[XY(x, 10)] = CRGB(70, 255, 0);
leds[XY(x, 14)] = CRGB(70, 255, 0);
for (x = 7; x < 12; x++) {
    leds[XY(x, 9)] = CRGB(70, 255, 0);
    leds[XY(x, 10)] = CRGB(70, 255, 0);
    leds[XY(x, 11)] = CRGB(70, 255, 0);
    leds[XY(x, 14)] = CRGB(70, 255, 0);
}
x = 12;
leds[XY(x, 10)] = CRGB(70, 255, 0);
leds[XY(x, 11)] = CRGB(70, 255, 0);
leds[XY(x, 14)] = CRGB(70, 255, 0);
x = 13;
leds[XY(x, 10)] = CRGB(70, 255, 0);

x = 1;
leds[XY(x, 12)] = CRGB(0, 255, 0);
leds[XY(x, 13)] = CRGB(0, 255, 0);
x = 2;
leds[XY(x, 9)] = CRGB(0, 255, 0);
leds[XY(x, 12)] = CRGB(0, 255, 0);
leds[XY(x, 14)] = CRGB(0, 255, 0);
x = 3;
leds[XY(x, 12)] = CRGB(0, 255, 0);
x = 4;
leds[XY(x, 12)] = CRGB(0, 255, 0);
x = 5;
leds[XY(x, 8)] = CRGB(0, 255, 0);
leds[XY(x, 12)] = CRGB(0, 255, 0);
x = 7;
leds[XY(x, 11)] = CRGB(0, 255, 0);
leds[XY(x, 13)] = CRGB(0, 255, 0);
for (x = 7; x < 12; x++) leds[XY(x, 13)] = CRGB(70, 255, 0);
x = 12;
leds[XY(x, 9)] = CRGB(0, 255, 0);
leds[XY(x, 13)] = CRGB(0, 255, 0);
x = 13;
leds[XY(x, 11)] = CRGB(0, 255, 0);
leds[XY(x, 14)] = CRGB(0, 255, 0);

```

```

    x = 14;
    leds[XY(x, 10)] = CRGB(0, 255, 0);
}

```

```

void krkhada1() {
    int x = 2;
    leds[XY(x, 6)] = CRGB(0, 80, 0);
    leds[XY(x, 7)] = CRGB(0, 80, 0);
    x = 3;
    leds[XY(x, 4)] = CRGB(0, 80, 0);
    leds[XY(x, 5)] = CRGB(0, 80, 0);
    x = 4;
    leds[XY(x, 3)] = CRGB(0, 80, 0);
    leds[XY(x, 4)] = CRGB(0, 80, 0);
    x = 5;
    leds[XY(x, 3)] = CRGB(0, 80, 0);
    leds[XY(x, 7)] = CRGB(0, 80, 0);
    x = 6;
    leds[XY(x, 2)] = CRGB(0, 80, 0);
    leds[XY(x, 6)] = CRGB(0, 80, 0);
    leds[XY(x, 7)] = CRGB(0, 80, 0);
    x = 7;
    leds[XY(x, 2)] = CRGB(0, 80, 0);
    leds[XY(x, 5)] = CRGB(0, 80, 0);
    x = 3;
    leds[XY(x, 7)] = CRGB(70, 255, 0);
    x = 7;
    leds[XY(x, 3)] = CRGB(70, 255, 0);
    leds[XY(x, 4)] = CRGB(70, 255, 0);
    x = 3;
    leds[XY(x, 6)] = CRGB(0, 255, 0);
    x = 4;
    leds[XY(x, 5)] = CRGB(0, 255, 0);
    leds[XY(x, 6)] = CRGB(0, 255, 0);
    leds[XY(x, 7)] = CRGB(0, 255, 0);
    x = 5;
    leds[XY(x, 4)] = CRGB(0, 255, 0);
    leds[XY(x, 5)] = CRGB(0, 255, 0);
    leds[XY(x, 6)] = CRGB(0, 255, 0);
    x = 6;
    leds[XY(x, 3)] = CRGB(0, 255, 0);
    leds[XY(x, 4)] = CRGB(0, 255, 0);
    leds[XY(x, 5)] = CRGB(0, 255, 0);
}

```

```

void krkhada2() {
    int x = 2;
    leds[XY(x, 6)] = CRGB(0, 80, 0);
    leds[XY(x, 7)] = CRGB(0, 80, 0);
}

```

```

x = 3;
leds[XY(x, 5)] = CRGB(0, 80, 0);
x = 4;
leds[XY(x, 4)] = CRGB(0, 80, 0);
x = 6;
leds[XY(x, 7)] = CRGB(0, 80, 0);
x = 7;
leds[XY(x, 7)] = CRGB(0, 80, 0);
x = 3;
leds[XY(x, 7)] = CRGB(70, 255, 0);
x = 3;
leds[XY(x, 6)] = CRGB(0, 255, 0);
x = 4;
leds[XY(x, 5)] = CRGB(0, 255, 0);
leds[XY(x, 6)] = CRGB(0, 255, 0);
leds[XY(x, 7)] = CRGB(0, 255, 0);
x = 5;
leds[XY(x, 5)] = CRGB(0, 255, 0);
leds[XY(x, 6)] = CRGB(0, 255, 0);
leds[XY(x, 7)] = CRGB(0, 255, 0);
x = 6;
leds[XY(x, 5)] = CRGB(0, 255, 0);
leds[XY(x, 6)] = CRGB(0, 255, 0);
}

```

```

void hlavahada1() {
  int x = 8;
  leds[XY(x, 1)] = CRGB(0, 80, 0);
  leds[XY(x, 5)] = CRGB(0, 80, 0);
  x = 9;
  leds[XY(x, 1)] = CRGB(0, 80, 0);
  leds[XY(x, 6)] = CRGB(0, 80, 0);
  x = 10;
  leds[XY(x, 1)] = CRGB(0, 80, 0);
  leds[XY(x, 6)] = CRGB(0, 80, 0);
  x = 11;
  leds[XY(x, 2)] = CRGB(0, 80, 0);
  leds[XY(x, 4)] = CRGB(0, 80, 0);
  x = 12;
  leds[XY(x, 3)] = CRGB(0, 80, 0);
  leds[XY(x, 4)] = CRGB(0, 80, 0);
  x = 8;
  leds[XY(x, 3)] = CRGB(70, 255, 0);
  leds[XY(x, 4)] = CRGB(70, 255, 0);
  x = 9;
  leds[XY(x, 2)] = CRGB(70, 255, 0);
  leds[XY(x, 3)] = CRGB(70, 255, 0);
  leds[XY(x, 4)] = CRGB(70, 255, 0);
  x = 10;
}

```

```

    leds[XY(x, 2)] = CRGB(70, 255, 0);
    leds[XY(x, 3)] = CRGB(70, 255, 0);
    leds[XY(x, 4)] = CRGB(70, 255, 0);
    x = 11;
    leds[XY(x, 3)] = CRGB(70, 255, 0);
    x = 9;
    leds[XY(x, 5)] = CRGB(0, 255, 0);
    x = 8;
    leds[XY(x, 2)] = CRGB(162, 255, 196);
    x = 12;
    leds[XY(x, 2)] = CRGB(162, 255, 196);
    x = 10;
    leds[XY(x, 5)] = CRGB(133, 0, 0);
    x = 11;
    leds[XY(x, 6)] = CRGB(255, 0, 0);
}

void hlavahada2() {
    int x = 8;
    leds[XY(x, 5)] = CRGB(0, 80, 0);
    x = 9;
    leds[XY(x, 1)] = CRGB(0, 80, 0);
    leds[XY(x, 4)] = CRGB(0, 80, 0);
    leds[XY(x, 5)] = CRGB(0, 80, 0);
    leds[XY(x, 6)] = CRGB(0, 80, 0);
    x = 10;
    leds[XY(x, 0)] = CRGB(0, 80, 0);
    leds[XY(x, 3)] = CRGB(0, 80, 0);
    leds[XY(x, 6)] = CRGB(0, 80, 0);
    x = 11;
    leds[XY(x, 0)] = CRGB(0, 80, 0);
    leds[XY(x, 3)] = CRGB(0, 80, 0);
    leds[XY(x, 6)] = CRGB(0, 80, 0);
    x = 12;
    leds[XY(x, 1)] = CRGB(0, 80, 0);
    leds[XY(x, 2)] = CRGB(0, 80, 0);
    x = 8;
    leds[XY(x, 2)] = CRGB(70, 255, 0);
    leds[XY(x, 3)] = CRGB(70, 255, 0);
    leds[XY(x, 4)] = CRGB(70, 255, 0);
    x = 9;
    leds[XY(x, 2)] = CRGB(70, 255, 0);
    leds[XY(x, 3)] = CRGB(70, 255, 0);
    x = 10;
    leds[XY(x, 1)] = CRGB(70, 255, 0);
    leds[XY(x, 2)] = CRGB(70, 255, 0);
    x = 11;
    leds[XY(x, 1)] = CRGB(70, 255, 0);
    x = 11;

```

```

    leds[XY(x, 2)] = CRGB(0, 255, 0);
    x = 8;
    leds[XY(x, 1)] = CRGB(162, 255, 196);
    x = 12;
    leds[XY(x, 0)] = CRGB(162, 255, 196);
    x = 10;
    leds[XY(x, 4)] = CRGB(133, 0, 0);
    leds[XY(x, 5)] = CRGB(133, 0, 0);
    x = 11;
    leds[XY(x, 5)] = CRGB(255, 0, 0);
    x = 12;
    leds[XY(x, 5)] = CRGB(255, 0, 0);
    x = 13;
    leds[XY(x, 6)] = CRGB(255, 0, 0);
}

```

```

void hlavahada3() {
    int x = 5;
    leds[XY(x, 3)] = CRGB(0, 80, 0);
    x = 6;
    leds[XY(x, 3)] = CRGB(0, 80, 0);
    x = 7;
    leds[XY(x, 3)] = CRGB(0, 80, 0);
    leds[XY(x, 6)] = CRGB(0, 80, 0);
    for (x = 8; x <= 11; x++) {
        leds[XY(x, 2)] = CRGB(0, 80, 0);
        leds[XY(x, 6)] = CRGB(0, 80, 0);
    }
    x = 12;
    leds[XY(x, 3)] = CRGB(0, 80, 0);
    leds[XY(x, 6)] = CRGB(0, 80, 0);
    x = 13;
    leds[XY(x, 4)] = CRGB(0, 80, 0);
    leds[XY(x, 5)] = CRGB(0, 80, 0);
    x = 8;
    leds[XY(x, 4)] = CRGB(70, 255, 0);
    leds[XY(x, 5)] = CRGB(70, 255, 0);
    x = 9;
    leds[XY(x, 4)] = CRGB(70, 255, 0);
    leds[XY(x, 5)] = CRGB(70, 255, 0);
    x = 10;
    leds[XY(x, 3)] = CRGB(70, 255, 0);
    leds[XY(x, 4)] = CRGB(70, 255, 0);
    leds[XY(x, 5)] = CRGB(70, 255, 0);
    x = 11;
    leds[XY(x, 3)] = CRGB(70, 255, 0);
    leds[XY(x, 4)] = CRGB(70, 255, 0);
    leds[XY(x, 5)] = CRGB(70, 255, 0);
    x = 12;
}

```

```

    leds[XY(x, 4)] = CRGB(70, 255, 0);
    leds[XY(x, 5)] = CRGB(70, 255, 0);
    x = 5;
    leds[XY(x, 4)] = CRGB(0, 255, 0);
    x = 6;
    leds[XY(x, 4)] = CRGB(0, 255, 0);
    x = 7;
    leds[XY(x, 4)] = CRGB(0, 255, 0);
    leds[XY(x, 5)] = CRGB(0, 255, 0);
    x = 8;
    leds[XY(x, 3)] = CRGB(0, 255, 0);
    x = 9;
    leds[XY(x, 3)] = CRGB(162, 255, 196);
    x = 13;
    leds[XY(x, 3)] = CRGB(162, 255, 196);
}

```

```

void hlavahada4() {
    int x = 5;
    leds[XY(x, 4)] = CRGB(0, 80, 0);
    x = 6;
    leds[XY(x, 3)] = CRGB(0, 80, 0);
    x = 7;
    leds[XY(x, 3)] = CRGB(0, 80, 0);
    for (x = 8; x <= 11; x++) {
        leds[XY(x, 3)] = CRGB(0, 80, 0);
        leds[XY(x, 7)] = CRGB(0, 80, 0);
    }
    x = 12;
    leds[XY(x, 4)] = CRGB(0, 80, 0);
    leds[XY(x, 7)] = CRGB(0, 80, 0);
    x = 13;
    leds[XY(x, 5)] = CRGB(0, 80, 0);
    leds[XY(x, 6)] = CRGB(0, 80, 0);
    x = 8;
    leds[XY(x, 5)] = CRGB(70, 255, 0);
    x = 9;
    leds[XY(x, 5)] = CRGB(70, 255, 0);
    leds[XY(x, 6)] = CRGB(70, 255, 0);
    x = 10;
    leds[XY(x, 4)] = CRGB(70, 255, 0);
    leds[XY(x, 5)] = CRGB(70, 255, 0);
    leds[XY(x, 6)] = CRGB(70, 255, 0);
    x = 11;
    leds[XY(x, 4)] = CRGB(70, 255, 0);
    leds[XY(x, 5)] = CRGB(70, 255, 0);
    leds[XY(x, 6)] = CRGB(70, 255, 0);
    x = 12;
    leds[XY(x, 5)] = CRGB(70, 255, 0);
}

```



```

    leds[XY(x, 6)] = CRGB(70, 255, 0);
    x = 6;
    leds[XY(x, 4)] = CRGB(0, 255, 0);
    x = 7;
    leds[XY(x, 4)] = CRGB(0, 255, 0);
    leds[XY(x, 5)] = CRGB(0, 255, 0);
    leds[XY(x, 6)] = CRGB(0, 255, 0);
    x = 8;
    leds[XY(x, 4)] = CRGB(0, 255, 0);
    leds[XY(x, 6)] = CRGB(0, 255, 0);
    x = 9;
    leds[XY(x, 4)] = CRGB(162, 255, 196);
    x = 13;
    leds[XY(x, 4)] = CRGB(162, 255, 196);
}

void mainanimacehada() {
    for (int i = 0; i < NUM_LEDS; i++) leds[i] = CRGB(0, 15, 15);
    if (animacehada == 0) {
        telohada2();
        krkhada2();
        hlavahada4();
        animacehada++;
    } else if (animacehada == 1) {
        telohada2();
        krkhada2();
        hlavahada3();
        animacehada++;
    } else if (animacehada == 2) {
        telohada1();
        krkhada1();
        hlavahada1();
        animacehada++;
    } else if (animacehada == 3) {
        telohada1();
        krkhada1();
        hlavahada2();
        animacehada++;
    } else if (animacehada == 4) {
        telohada1();
        krkhada1();
        hlavahada1();
        animacehada++;
    } else if (animacehada == 5) {
        telohada2();
        krkhada2();
        hlavahada3();
        animacehada++;
    } else if (animacehada == 6) {

```

```

        telohada2();
        krkhada2();
        hlavahada4();
        animacehada++;
    }
    if (animacehada >= 6) animacehada = 0;
}

// ===== MIKROFON ikona =====

void animacemikrofon() {
    for (int i = 0; i < NUM_LEDS; i++) leds[i] = CRGB(15, 0, 15);
    int x = 0;
    leds[XY(x, 7)] = CRGB(255, 255, 255);
    x = 1;
    leds[XY(x, 7)] = CRGB(255, 255, 255);
    leds[XY(x, 6)] = CRGB(255, 255, 255);
    leds[XY(x, 5)] = CRGB(255, 255, 255);
    leds[XY(x, 4)] = CRGB(255, 255, 255);
    x = 2;
    leds[XY(x, 5)] = CRGB(255, 255, 255);
    x = 4;
    leds[XY(x, 4)] = CRGB(255, 255, 255);
    leds[XY(x, 9)] = CRGB(255, 255, 255);
    leds[XY(x, 10)] = CRGB(255, 255, 255);
    x = 5;
    leds[XY(x, 4)] = CRGB(255, 255, 255);
    leds[XY(x, 3)] = CRGB(255, 255, 255);
    leds[XY(x, 2)] = CRGB(255, 255, 255);
    leds[XY(x, 1)] = CRGB(255, 255, 255);
    leds[XY(x, 8)] = CRGB(255, 255, 255);
    leds[XY(x, 9)] = CRGB(255, 255, 255);
    x = 6;
    leds[XY(x, 2)] = CRGB(255, 255, 255);
    leds[XY(x, 7)] = CRGB(255, 255, 255);
    leds[XY(x, 8)] = CRGB(255, 255, 255);
    x = 7;
    leds[XY(x, 6)] = CRGB(255, 255, 255);
    leds[XY(x, 7)] = CRGB(255, 255, 255);
    x = 10;
    leds[XY(x, 2)] = CRGB(255, 255, 255);
    x = 11;
    leds[XY(x, 1)] = CRGB(255, 255, 255);
    leds[XY(x, 2)] = CRGB(255, 255, 255);
    leds[XY(x, 3)] = CRGB(255, 255, 255);
    x = 12;
    leds[XY(x, 2)] = CRGB(255, 255, 255);
    x = 4;
    leds[XY(x, 11)] = CRGB(130, 130, 130);
}

```

```

x = 5;
leds[XY(x, 11)] = CRGB(130, 130, 130);
leds[XY(x, 10)] = CRGB(130, 130, 130);
x = 6;
leds[XY(x, 11)] = CRGB(130, 130, 130);
x = 7;
leds[XY(x, 10)] = CRGB(130, 130, 130);
x = 8;
for (int i = 5; i <= 9; i++) leds[XY(x, i)] = CRGB(130, 130, 130);
x = 9;
leds[XY(x, 8)] = CRGB(130, 130, 130);
leds[XY(x, 7)] = CRGB(130, 130, 130);
leds[XY(x, 6)] = CRGB(130, 130, 130);
leds[XY(x, 2)] = CRGB(130, 130, 130);
x = 10;
leds[XY(x, 7)] = CRGB(130, 130, 130);
leds[XY(x, 3)] = CRGB(130, 130, 130);
leds[XY(x, 1)] = CRGB(130, 130, 130);
x = 11;
leds[XY(x, 4)] = CRGB(130, 130, 130);
leds[XY(x, 0)] = CRGB(130, 130, 130);
x = 12;
leds[XY(x, 3)] = CRGB(130, 130, 130);
leds[XY(x, 1)] = CRGB(130, 130, 130);
x = 13;
leds[XY(x, 2)] = CRGB(130, 130, 130);
x = 3;
for (int i = 9; i <= 13; i++) leds[XY(x, i)] = CRGB(80, 80, 80);
x = 4;
leds[XY(x, 14)] = CRGB(80, 80, 80);
leds[XY(x, 12)] = CRGB(80, 80, 80);
leds[XY(x, 8)] = CRGB(80, 80, 80);
x = 5;
leds[XY(x, 15)] = CRGB(80, 80, 80);
leds[XY(x, 12)] = CRGB(80, 80, 80);
leds[XY(x, 7)] = CRGB(80, 80, 80);
x = 6;
leds[XY(x, 15)] = CRGB(80, 80, 80);
leds[XY(x, 12)] = CRGB(80, 80, 80);
leds[XY(x, 10)] = CRGB(80, 80, 80);
leds[XY(x, 9)] = CRGB(80, 80, 80);
leds[XY(x, 6)] = CRGB(80, 80, 80);
x = 7;
leds[XY(x, 15)] = CRGB(80, 80, 80);
leds[XY(x, 11)] = CRGB(80, 80, 80);
leds[XY(x, 9)] = CRGB(80, 80, 80);
leds[XY(x, 8)] = CRGB(80, 80, 80);
leds[XY(x, 5)] = CRGB(80, 80, 80);
x = 8;

```

```

    leds[XY(x, 14)] = CRGB(80, 80, 80);
    leds[XY(x, 10)] = CRGB(80, 80, 80);
    for (int i = 2; i <= 4; i++) leds[XY(x, i)] = CRGB(80, 80, 80);
    x = 9;
    leds[XY(x, 13)] = CRGB(80, 80, 80);
    leds[XY(x, 9)] = CRGB(80, 80, 80);
    leds[XY(x, 5)] = CRGB(80, 80, 80);
    leds[XY(x, 4)] = CRGB(80, 80, 80);
    leds[XY(x, 3)] = CRGB(80, 80, 80);
    leds[XY(x, 1)] = CRGB(80, 80, 80);
    x = 10;
    leds[XY(x, 12)] = CRGB(80, 80, 80);
    leds[XY(x, 8)] = CRGB(80, 80, 80);
    leds[XY(x, 6)] = CRGB(80, 80, 80);
    leds[XY(x, 5)] = CRGB(80, 80, 80);
    leds[XY(x, 4)] = CRGB(80, 80, 80);
    leds[XY(x, 0)] = CRGB(80, 80, 80);
    x = 11;
    leds[XY(x, 11)] = CRGB(80, 80, 80);
    leds[XY(x, 7)] = CRGB(80, 80, 80);
    leds[XY(x, 6)] = CRGB(80, 80, 80);
    leds[XY(x, 5)] = CRGB(80, 80, 80);
    x = 12;
    leds[XY(x, 11)] = CRGB(80, 80, 80);
    leds[XY(x, 0)] = CRGB(80, 80, 80);
    for (int i = 4; i <= 7; i++) leds[XY(x, i)] = CRGB(80, 80, 80);
    x = 13;
    leds[XY(x, 11)] = CRGB(80, 80, 80);
    leds[XY(x, 15)] = CRGB(80, 80, 80);
    leds[XY(x, 1)] = CRGB(80, 80, 80);
    leds[XY(x, 0)] = CRGB(80, 80, 80);
    for (int i = 3; i <= 7; i++) leds[XY(x, i)] = CRGB(80, 80, 80);
    x = 14;
    for (int i = 12; i <= 14; i++) leds[XY(x, i)] = CRGB(80, 80, 80);
    for (int i = 1; i <= 6; i++) leds[XY(x, i)] = CRGB(80, 80, 80);
    x = 15;
    for (int i = 2; i <= 5; i++) leds[XY(x, i)] = CRGB(80, 80, 80);
}

```

// ===== TUŽKA ikona =====

```

void tuzka() {
    for (int i = 0; i < NUM_LEDS; i++) leds[i] = CRGB(0, 15, 15);
    int x = 4;
    leds[XY(x, 8)] = CRGB(255, 180, 120);
    leds[XY(x, 9)] = CRGB(255, 180, 120);
    x = 5;
    leds[XY(x, 9)] = CRGB(255, 180, 120);
    leds[XY(x, 10)] = CRGB(255, 180, 120);
}

```

```

x = 6;
leds[XY(x, 10)] = CRGB(255, 180, 120);
leds[XY(4, 10)] = CRGB(0, 0, 0);
leds[XY(12, 11)] = CRGB(0, 0, 0);
leds[XY(13, 11)] = CRGB(0, 0, 0);
leds[XY(4, 12)] = CRGB(0, 0, 0);
leds[XY(5, 12)] = CRGB(0, 0, 0);
leds[XY(6, 12)] = CRGB(0, 0, 0);
leds[XY(7, 12)] = CRGB(0, 0, 0);
leds[XY(10, 12)] = CRGB(0, 0, 0);
leds[XY(11, 12)] = CRGB(0, 0, 0);
leds[XY(14, 12)] = CRGB(0, 0, 0);
leds[XY(8, 13)] = CRGB(0, 0, 0);
leds[XY(9, 13)] = CRGB(0, 0, 0);
x = 4;
leds[XY(x, 7)] = CRGB(100, 100, 0);
x = 5;
leds[XY(x, 7)] = CRGB(100, 100, 0);
leds[XY(x, 6)] = CRGB(100, 100, 0);
x = 6;
leds[XY(x, 6)] = CRGB(100, 100, 0);
leds[XY(x, 5)] = CRGB(100, 100, 0);
x = 7;
leds[XY(x, 10)] = CRGB(100, 100, 0);
leds[XY(x, 9)] = CRGB(100, 100, 0);
leds[XY(x, 5)] = CRGB(100, 100, 0);
leds[XY(x, 4)] = CRGB(100, 100, 0);
x = 8;
leds[XY(x, 9)] = CRGB(100, 100, 0);
leds[XY(x, 8)] = CRGB(100, 100, 0);
leds[XY(x, 4)] = CRGB(100, 100, 0);
leds[XY(x, 3)] = CRGB(100, 100, 0);
x = 9;
leds[XY(x, 8)] = CRGB(100, 100, 0);
leds[XY(x, 7)] = CRGB(100, 100, 0);
leds[XY(x, 3)] = CRGB(100, 100, 0);
x = 10;
leds[XY(x, 6)] = CRGB(100, 100, 0);
leds[XY(x, 7)] = CRGB(100, 100, 0);
x = 11;
leds[XY(x, 6)] = CRGB(100, 100, 0);
leds[XY(x, 5)] = CRGB(100, 100, 0);
x = 5;
leds[XY(x, 8)] = CRGB(255, 255, 0);
x = 6;
leds[XY(x, 7)] = CRGB(255, 255, 0);
leds[XY(x, 8)] = CRGB(255, 255, 0);
leds[XY(x, 9)] = CRGB(255, 255, 0);
x = 7;

```

```

    leds[XY(x, 6)] = CRGB(255, 255, 0);
    leds[XY(x, 7)] = CRGB(255, 255, 0);
    leds[XY(x, 8)] = CRGB(255, 255, 0);
    x = 8;
    leds[XY(x, 5)] = CRGB(255, 255, 0);
    leds[XY(x, 6)] = CRGB(255, 255, 0);
    leds[XY(x, 7)] = CRGB(255, 255, 0);
    x = 9;
    leds[XY(x, 4)] = CRGB(255, 255, 0);
    leds[XY(x, 5)] = CRGB(255, 255, 0);
    leds[XY(x, 6)] = CRGB(255, 255, 0);
    x = 10;
    leds[XY(x, 4)] = CRGB(255, 255, 0);
    leds[XY(x, 5)] = CRGB(255, 255, 0);
    x = 10;
    leds[XY(x, 3)] = CRGB(255, 255, 255);
    x = 11;
    leds[XY(x, 2)] = CRGB(255, 255, 255);
    leds[XY(x, 3)] = CRGB(255, 255, 255);
    leds[XY(x, 4)] = CRGB(255, 255, 255);
    x = 12;
    leds[XY(x, 2)] = CRGB(255, 255, 255);
    leds[XY(x, 3)] = CRGB(255, 255, 255);
    x = 9;
    leds[XY(x, 2)] = CRGB(80, 80, 80);
    x = 10;
    leds[XY(x, 1)] = CRGB(80, 80, 80);
    leds[XY(x, 2)] = CRGB(80, 80, 80);
    x = 11;
    leds[XY(x, 1)] = CRGB(80, 80, 80);
    x = 12;
    leds[XY(x, 4)] = CRGB(80, 80, 80);
    leds[XY(x, 5)] = CRGB(80, 80, 80);
    x = 13;
    leds[XY(x, 3)] = CRGB(80, 80, 80);
    leds[XY(x, 4)] = CRGB(80, 80, 80);
    x = 12;
    leds[XY(x, 1)] = CRGB(255, 0, 0);
    x = 13;
    leds[XY(x, 1)] = CRGB(255, 0, 0);
    leds[XY(x, 2)] = CRGB(255, 0, 0);
    x = 11;
    leds[XY(x, 0)] = CRGB(100, 0, 0);
    x = 12;
    leds[XY(x, 0)] = CRGB(100, 0, 0);
    x = 14;
    leds[XY(x, 2)] = CRGB(100, 0, 0);
    leds[XY(x, 3)] = CRGB(100, 0, 0);
}

```

```
// ===== PACMAN =====
```

```
void pacman1() {
    for (int i = 5; i <= 10; i++) {
        leds[XY(1, i)] = CRGB(255, 255, 0);
        leds[XY(14, i)] = CRGB(255, 255, 0);
    }
    for (int i = 3; i <= 12; i++) {
        leds[XY(2, i)] = CRGB(255, 255, 0);
        leds[XY(13, i)] = CRGB(255, 255, 0);
    }
    for (int i = 2; i <= 13; i++) {
        leds[XY(3, i)] = CRGB(255, 255, 0);
        leds[XY(4, i)] = CRGB(255, 255, 0);
        leds[XY(11, i)] = CRGB(255, 255, 0);
        leds[XY(12, i)] = CRGB(255, 255, 0);
    }
    for (int i = 1; i <= 14; i++) {
        leds[XY(5, i)] = CRGB(255, 255, 0);
        leds[XY(6, i)] = CRGB(255, 255, 0);
        leds[XY(7, i)] = CRGB(255, 255, 0);
        leds[XY(8, i)] = CRGB(255, 255, 0);
        leds[XY(9, i)] = CRGB(255, 255, 0);
        leds[XY(10, i)] = CRGB(255, 255, 0);
    }
}

void pacman2() {
    for (int i = 5; i <= 10; i++) leds[XY(1, i)] = CRGB(255, 255, 0);
    for (int i = 3; i <= 12; i++) leds[XY(2, i)] = CRGB(255, 255, 0);
    for (int i = 2; i <= 13; i++) {
        leds[XY(3, i)] = CRGB(255, 255, 0);
        leds[XY(4, i)] = CRGB(255, 255, 0);
    }
    for (int i = 1; i <= 14; i++) leds[XY(5, i)] = CRGB(255, 255, 0);
    for (int x = 6; x <= 7; x++) {
        for (int i = 1; i <= 6; i++) leds[XY(x, i)] = CRGB(255, 255, 0);
        for (int i = 8; i <= 14; i++) leds[XY(x, i)] = CRGB(255, 255, 0);
    }
    int p = 8;
    for (int i = 1; i <= 5; i++) leds[XY(p, i)] = CRGB(255, 255, 0);
    for (int i = 9; i <= 14; i++) leds[XY(p, i)] = CRGB(255, 255, 0);
    p = 9;
    for (int i = 1; i <= 5; i++) leds[XY(p, i)] = CRGB(255, 255, 0);
    for (int i = 9; i <= 14; i++) leds[XY(p, i)] = CRGB(255, 255, 0);
    p = 10;
    for (int i = 1; i <= 5; i++) leds[XY(p, i)] = CRGB(255, 255, 0);
    for (int i = 9; i <= 14; i++) leds[XY(p, i)] = CRGB(255, 255, 0);
}
```

```

p = 11;
for (int i = 2; i <= 5; i++) leds[XY(p, i)] = CRGB(255, 255, 0);
for (int i = 9; i <= 13; i++) leds[XY(p, i)] = CRGB(255, 255, 0);
p = 12;
for (int i = 2; i <= 5; i++) leds[XY(p, i)] = CRGB(255, 255, 0);
for (int i = 9; i <= 13; i++) leds[XY(p, i)] = CRGB(255, 255, 0);
p = 13;
for (int i = 3; i <= 4; i++) leds[XY(p, i)] = CRGB(255, 255, 0);
for (int i = 10; i <= 12; i++) leds[XY(p, i)] = CRGB(255, 255, 0);
p = 14;
leds[XY(p, 4)] = CRGB(255, 255, 0);
leds[XY(p, 10)] = CRGB(255, 255, 0);
}

void pacman3() {
    for (int i = 5; i <= 10; i++) leds[XY(1, i)] = CRGB(255, 255, 0);
    for (int i = 3; i <= 12; i++) leds[XY(2, i)] = CRGB(255, 255, 0);
    for (int i = 2; i <= 13; i++) {
        leds[XY(3, i)] = CRGB(255, 255, 0);
        leds[XY(4, i)] = CRGB(255, 255, 0);
    }
    for (int i = 1; i <= 14; i++) leds[XY(5, i)] = CRGB(255, 255, 0);
    for (int x = 6; x <= 7; x++) {
        for (int i = 1; i <= 6; i++) leds[XY(x, i)] = CRGB(255, 255, 0);
        for (int i = 9; i <= 14; i++) leds[XY(x, i)] = CRGB(255, 255, 0);
    }
    int p = 8;
    for (int i = 1; i <= 5; i++) leds[XY(p, i)] = CRGB(255, 255, 0);
    for (int i = 10; i <= 14; i++) leds[XY(p, i)] = CRGB(255, 255, 0);
    p = 9;
    for (int i = 1; i <= 5; i++) leds[XY(p, i)] = CRGB(255, 255, 0);
    for (int i = 10; i <= 14; i++) leds[XY(p, i)] = CRGB(255, 255, 0);
    p = 10;
    for (int i = 1; i <= 4; i++) leds[XY(p, i)] = CRGB(255, 255, 0);
    for (int i = 11; i <= 14; i++) leds[XY(p, i)] = CRGB(255, 255, 0);
    p = 11;
    for (int i = 2; i <= 4; i++) leds[XY(p, i)] = CRGB(255, 255, 0);
    for (int i = 11; i <= 13; i++) leds[XY(p, i)] = CRGB(255, 255, 0);
    p = 12;
    for (int i = 2; i <= 3; i++) leds[XY(p, i)] = CRGB(255, 255, 0);
    for (int i = 12; i <= 13; i++) leds[XY(p, i)] = CRGB(255, 255, 0);
    p = 13;
    leds[XY(p, 3)] = CRGB(255, 255, 0);
    leds[XY(p, 12)] = CRGB(255, 255, 0);
}

void pacman() {
    for (int i = 0; i < NUM_LEDS; i++) leds[i] = CRGB(0, 0, 15);
    if (poradianimace == 0) {

```



```

    pacman1();
    poradianimace++;
} else if (poradianimace == 1) {
    pacman2();
    poradianimace++;
} else if (poradianimace == 2) {
    pacman3();
    poradianimace++;
} else if (poradianimace == 3) {
    pacman2();
    poradianimace++;
}
if (poradianimace > 3) poradianimace = 0;
}

// ===== WIFI JOIN obrazovka =====
void joinwifi() {
    vykreslicislo(1, 5, 1, 255, 255, 255);
    vykreslicislo(5, 5, 0, 255, 255, 255);
    vykreslicislo(7, 5, 1, 255, 255, 255);
    vykreslicislo(9, 5, 1, 255, 255, 255);
    vykreslicislo(12, 5, 1, 255, 255, 255);
    vykreslicislo(5, 13, 1, 255, 255, 255);
    vykreslicislo(7, 13, 1, 255, 255, 255);
    vykreslicislo(11, 13, 1, 255, 255, 255);
    int x = 1;
    for (int i = 4; i <= 5; i++) leds[XY(x, i)] = CRGB(255, 255, 255);
    for (int i = 9; i <= 11; i++) leds[XY(x, i)] = CRGB(255, 255, 255);
    x = 2;
    for (int i = 12; i <= 13; i++) leds[XY(x, i)] = CRGB(255, 255, 255);
    leds[XY(x, 5)] = CRGB(255, 255, 255);
    x = 3;
    for (int i = 10; i <= 11; i++) leds[XY(x, i)] = CRGB(255, 255, 255);
    x = 4;
    for (int i = 12; i <= 13; i++) leds[XY(x, i)] = CRGB(255, 255, 255);
    x = 5;
    for (int i = 9; i <= 11; i++) leds[XY(x, i)] = CRGB(255, 255, 255);
    x = 10;
    leds[XY(x, 9)] = CRGB(255, 255, 255);
    leds[XY(x, 11)] = CRGB(255, 255, 255);
    x = 11;
    leds[XY(x, 9)] = CRGB(255, 255, 255);
    leds[XY(x, 11)] = CRGB(255, 255, 255);
    x = 12;
    leds[XY(x, 2)] = CRGB(255, 255, 255);
    x = 13;
    leds[XY(x, 3)] = CRGB(255, 255, 255);
    leds[XY(x, 4)] = CRGB(255, 255, 255);
}

```

3.6 Sprites.h

```
#pragma once
#include <Arduino.h>

void telohada1();
void telohada2();
void krkhada1();
void krkhada2();
void hlavahada1();
void hlavahada2();
void hlavahada3();
void hlavahada4();
void mainanimacehada();
void animacemikrofon();
void tuzka();
void pacman1();
void pacman2();
void pacman3();
void pacman();
void joinwifi();
```

4 Vzdálené kreslení

4.1 Drawing.cpp

```
#pragma once
#include "config.h"
#include "display_utils.h"
#include "buttons.h"
#include "clock_wifi.h"
#include "animations.h"
#include "snake.h"
#include "vumetr.h"
#include "drawing.h"
#include "sprites.h"
#include "audio_reactive.h"

const char MAIN_page[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
<meta name="viewport" content="width=device-width, initial-scale=1">
<style>
body { background:#000; color:#fff; font-family:sans-serif; text-align:center;
touch-action:none; }
```

```

canvas { border:2px solid #444; width:320px; height:320px; image-
rendering:pixelated; touch-action:none; }
.controls { margin-top:10px; }
button, input { background:#222; color:#fff; border:1px solid #555;
padding:8px; margin:4px; font-size:16px; }
</style>
</head>
<body>
<h2>RGB DRAW</h2>
<canvas id="c" width="16" height="16"></canvas>
<div class="controls">
<input type="color" id="color" value="#ff0000"><br>
<button onclick="setBrush(1)">1x1</button>
<button onclick="setBrush(3)">3x3</button>
<button onclick="setBrush(5)">5x5</button><br>
<button onclick="eraser()">Guma</button>
<button onclick="clearPanel()">Clear</button>
</div>
<script>
const canvas=document.getElementById("c");
const ctx=canvas.getContext("2d");
ctx.imageSmoothingEnabled=false;
let drawing=false,brush=1,erase=false;
function setBrush(s){brush=s;erase=false;}
function eraser(){erase=true;}
function hexToRgb(h){const
n=parseInt(h.slice(1),16);return[(n>>16)&255,(n>>8)&255,n&255];}
function draw(cx,cy){
  const r=canvas.getBoundingClientRect();
  const px=Math.floor((cx-r.left)/r.width*16),py=Math.floor((cy-
r.top)/r.height*16);
  const
half=Math.floor(brush/2),col=erase?[0,0,0]:hexToRgb(document.getElementById("c
olor").value);
  let pixels=[];
  for(let y=-half;y<=half;y++)for(let x=-half;x<=half;x++){
    const qx=px+x,qy=py+y;
    if(qx<0||qy<0||qx>=16||qy>=16)continue;
    ctx.fillStyle=erase?"#000":document.getElementById("color").value;
    ctx.fillRect(qx,qy,1,1);
    pixels.push({x:qx,y:qy,r:col[0],g:col[1],b:col[2]});
  }
  if(pixels.length)fetch("/update",{method:"POST",headers:{"Content-
Type":"application/json"},body:JSON.stringify({pixels})});
}
["mousedown","touchstart"].forEach(e=>canvas.addEventListener(e,ev=>{drawing=t
rue;const
p=ev.touches?ev.touches[0]:ev;draw(p.clientX,p.clientY);ev.preventDefault();})
);

```

```

["mousemove", "touchmove"].forEach(e=>canvas.addEventListener(e, ev=>{if(!drawing)return;const
p=ev.touches?ev.touches[0]:ev;draw(p.clientX,p.clientY);ev.preventDefault();})
);
["mouseup", "mouseleave", "touchend"].forEach(e=>canvas.addEventListener(e, ()=>drawing=false));
function
clearPanel(){ctx.fillStyle="#000";ctx.fillRect(0,0,16,16);fetch("/clear",{method:"POST"}});}
</script>
</body>
</html>
)rawliteral";

```

```

void handleRoot() {
    server.send_P(200, "text/html", MAIN_page);
}

void handleUpdate() {
    if (!server.hasArg("plain")) {
        server.send(400, "text/plain", "NO DATA");
        return;
    }
    DynamicJsonDocument doc(2048);
    if (deserializeJson(doc, server.arg("plain"))) {
        server.send(400, "text/plain", "BAD JSON");
        return;
    }
    for (JsonObject p : doc["pixels"].as<JsonArray>()) {
        uint8_t x = p["x"], y = p["y"], r = p["r"], g = p["g"], b = p["b"];
        if (x < WIDTH && y < HEIGHT) {
            ledBuffer[x][y] = CRGB(r, g, b);
            needsShow = true;
        }
    }
    server.send(200, "text/plain", "OK");
}

void handleClear() {
    for (uint8_t y = 0; y < HEIGHT; y++)
        for (uint8_t x = 0; x < WIDTH; x++) ledBuffer[x][y] = CRGB::Black;
    needsShow = true;
    server.send(200, "text/plain", "CLEARED");
}

void kresleni() {
    if (!kresleniActive) {
        kresleniActive = true;
        FastLED.clear(true);
    }
}

```

```

    delay(5);
    WiFi.disconnect(true);
    delay(100);
    WiFi.mode(WIFI_AP);
    WiFi.softAP("RGB-DISPLAY", "12345678");
    server.on("/", handleRoot);
    server.on("/update", HTTP_POST, handleUpdate);
    server.on("/clear", HTTP_POST, handleClear);
    server.begin();
    joinwifi();
    FastLED.show();
}

server.handleClient();
static unsigned long lastShow = 0;
if (millis() - lastShow > 50 && needsShow) {
    for (uint8_t y = 0; y < HEIGHT; y++)
        for (uint8_t x = 0; x < WIDTH; x++) leds[XY(x, y)] = ledBuffer[x][y];
    FastLED.show();
    needsShow = false;
    lastShow = millis();
}
}

void stopKresleni() {
    if (!kresleniActive) return;
    kresleniActive = false;
    server.stop();
    WiFi.softAPdisconnect(true);
    delay(100);
    WiFi.mode(WIFI_STA);
    WiFi.begin(ssid, password);
}

```

4.2 Drawing.h

```

#pragma once
#include <Arduino.h>

void kresleni();
void stopKresleni();
void handleRoot();
void handleUpdate();
void handleClear();

```

4.3 Display_utils.cpp

```
#pragma once
#include "config.h"
#include "display_utils.h"
#include "buttons.h"
#include "clock_wifi.h"
#include "animations.h"
#include "snake.h"
#include "vumetr.h"
#include "drawing.h"
#include "sprites.h"
#include "audio_reactive.h"

// Převod (x,y) → index pro had-uspořádanou matici
uint16_t XY(uint8_t x, uint8_t y) {
    if (y % 2 == 0) {
        return (y * WIDTH) + x;
    } else {
        return (y * WIDTH) + (WIDTH - 1 - x);
    }
}

// Vykreslení číslice 0-9 na pozici (X, Y) s danou barvou
void vykreslicislo(uint8_t X, uint8_t Y, uint8_t cislo, uint8_t R, uint8_t G,
uint8_t B) {
    if (cislo == 0) {
        leds[XY(X, Y)] = CRGB(R, G, B);
        leds[XY(X, Y - 1)] = CRGB(R, G, B);
        leds[XY(X, Y - 2)] = CRGB(R, G, B);
        leds[XY(X, Y - 3)] = CRGB(R, G, B);
        leds[XY(X, Y - 4)] = CRGB(R, G, B);
        leds[XY(X + 1, Y - 4)] = CRGB(R, G, B);
        leds[XY(X + 1, Y)] = CRGB(R, G, B);
        leds[XY(X + 2, Y)] = CRGB(R, G, B);
        leds[XY(X + 2, Y - 1)] = CRGB(R, G, B);
        leds[XY(X + 2, Y - 2)] = CRGB(R, G, B);
        leds[XY(X + 2, Y - 3)] = CRGB(R, G, B);
        leds[XY(X + 2, Y - 4)] = CRGB(R, G, B);
    }
    if (cislo == 1) {
        leds[XY(X + 2, Y)] = CRGB(R, G, B);
        leds[XY(X + 2, Y - 1)] = CRGB(R, G, B);
        leds[XY(X + 2, Y - 2)] = CRGB(R, G, B);
        leds[XY(X + 2, Y - 3)] = CRGB(R, G, B);
        leds[XY(X + 2, Y - 4)] = CRGB(R, G, B);
    }
    if (cislo == 2) {
```

```

    leds[XY(X, Y)] = CRGB(R, G, B);
    leds[XY(X, Y - 1)] = CRGB(R, G, B);
    leds[XY(X, Y - 2)] = CRGB(R, G, B);
    leds[XY(X, Y - 4)] = CRGB(R, G, B);
    leds[XY(X + 1, Y)] = CRGB(R, G, B);
    leds[XY(X + 1, Y - 2)] = CRGB(R, G, B);
    leds[XY(X + 1, Y - 4)] = CRGB(R, G, B);
    leds[XY(X + 2, Y)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 2)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 3)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 4)] = CRGB(R, G, B);
}
if (cislo == 3) {
    leds[XY(X + 2, Y)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 1)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 2)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 3)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 4)] = CRGB(R, G, B);
    leds[XY(X, Y)] = CRGB(R, G, B);
    leds[XY(X, Y - 2)] = CRGB(R, G, B);
    leds[XY(X, Y - 4)] = CRGB(R, G, B);
    leds[XY(X + 1, Y)] = CRGB(R, G, B);
    leds[XY(X + 1, Y - 2)] = CRGB(R, G, B);
    leds[XY(X + 1, Y - 4)] = CRGB(R, G, B);
}
if (cislo == 4) {
    leds[XY(X + 2, Y)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 1)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 2)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 3)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 4)] = CRGB(R, G, B);
    leds[XY(X, Y - 2)] = CRGB(R, G, B);
    leds[XY(X, Y - 3)] = CRGB(R, G, B);
    leds[XY(X, Y - 4)] = CRGB(R, G, B);
    leds[XY(X + 1, Y - 2)] = CRGB(R, G, B);
}
if (cislo == 5) {
    leds[XY(X, Y)] = CRGB(R, G, B);
    leds[XY(X, Y - 2)] = CRGB(R, G, B);
    leds[XY(X, Y - 3)] = CRGB(R, G, B);
    leds[XY(X, Y - 4)] = CRGB(R, G, B);
    leds[XY(X + 1, Y)] = CRGB(R, G, B);
    leds[XY(X + 1, Y - 2)] = CRGB(R, G, B);
    leds[XY(X + 1, Y - 4)] = CRGB(R, G, B);
    leds[XY(X + 2, Y)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 1)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 2)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 4)] = CRGB(R, G, B);
}

```

```

if (cislo == 6) {
    leds[XY(X, Y)] = CRGB(R, G, B);
    leds[XY(X, Y - 1)] = CRGB(R, G, B);
    leds[XY(X, Y - 2)] = CRGB(R, G, B);
    leds[XY(X, Y - 3)] = CRGB(R, G, B);
    leds[XY(X, Y - 4)] = CRGB(R, G, B);
    leds[XY(X + 1, Y)] = CRGB(R, G, B);
    leds[XY(X + 1, Y - 2)] = CRGB(R, G, B);
    leds[XY(X + 1, Y - 4)] = CRGB(R, G, B);
    leds[XY(X + 2, Y)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 1)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 2)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 4)] = CRGB(R, G, B);
}
if (cislo == 7) {
    leds[XY(X + 2, Y)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 1)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 2)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 3)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 4)] = CRGB(R, G, B);
    leds[XY(X, Y - 4)] = CRGB(R, G, B);
    leds[XY(X + 1, Y - 4)] = CRGB(R, G, B);
}
if (cislo == 8) {
    leds[XY(X, Y)] = CRGB(R, G, B);
    leds[XY(X, Y - 1)] = CRGB(R, G, B);
    leds[XY(X, Y - 2)] = CRGB(R, G, B);
    leds[XY(X, Y - 3)] = CRGB(R, G, B);
    leds[XY(X, Y - 4)] = CRGB(R, G, B);
    leds[XY(X + 1, Y)] = CRGB(R, G, B);
    leds[XY(X + 1, Y - 2)] = CRGB(R, G, B);
    leds[XY(X + 1, Y - 4)] = CRGB(R, G, B);
    leds[XY(X + 2, Y)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 1)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 2)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 3)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 4)] = CRGB(R, G, B);
}
if (cislo == 9) {
    leds[XY(X + 2, Y)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 1)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 2)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 3)] = CRGB(R, G, B);
    leds[XY(X + 2, Y - 4)] = CRGB(R, G, B);
    leds[XY(X, Y)] = CRGB(R, G, B);
    leds[XY(X, Y - 2)] = CRGB(R, G, B);
    leds[XY(X, Y - 3)] = CRGB(R, G, B);
    leds[XY(X, Y - 4)] = CRGB(R, G, B);
    leds[XY(X + 1, Y)] = CRGB(R, G, B);
}

```



```

        leds[XY(X + 1, Y - 2)] = CRGB(R, G, B);
        leds[XY(X + 1, Y - 4)] = CRGB(R, G, B);
    }
}

```

4.4 Display_utils.h

```

#pragma once
#include <Arduino.h>

uint16_t XY(uint8_t x, uint8_t y);
void vykreslicislo(uint8_t X, uint8_t Y, uint8_t cislo, uint8_t R, uint8_t G,
uint8_t B);

```

5 Čas

5.1 Clock_wifi.cpp

```

#pragma once
#include "config.h"
#include "display_utils.h"
#include "buttons.h"
#include "clock_wifi.h"
#include "animations.h"
#include "snake.h"
#include "vumetr.h"
#include "drawing.h"
#include "sprites.h"
#include "audio_reactive.h"

void setRTCfromNTP() {
    struct tm timeinfo;
    if (getLocalTime(&timeinfo)) {
        DateTime nowRTC(1900 + timeinfo.tm_year, 1 + timeinfo.tm_mon,
                        timeinfo.tm_mday, timeinfo.tm_hour,
                        timeinfo.tm_min, timeinfo.tm_sec);
        rtc.adjust(nowRTC);
    }
}

void manageWiFiNonBlocking() {
    unsigned long now = millis();
    const unsigned long wifiCheckInterval = 10000;
    const unsigned long wifiConnectTimeout = 10000;

    if (WiFi.status() != WL_CONNECTED) {
        if (!wifiConnecting && (now - lastWiFiCheck >= wifiCheckInterval)) {

```

```

    WiFi.begin(ssid, password);
    wifiConnecting = true;
    wifiConnectStart = now;
    lastWiFiCheck = now;
}
if (wifiConnecting && (now - wifiConnectStart) > wifiConnectTimeout) {
    WiFi.disconnect();
    delay(50);
    WiFi.begin(ssid, password);
    wifiConnectStart = now;
}
} else {
    if (wifiConnecting) {
        wifiConnecting = false;
        configTzTime("CET-1CEST,M3.5.0/2,M10.5.0/3", ntpServer);
        setRTCfromNTP();
    }
}
}

void nacticaszwifi() {
    const unsigned long timeInterval = 1000;
    unsigned long now = millis();
    uint8_t R = 255, G = 255, B = 255;

    manageWiFiNonBlocking();

    if (now - lastTimePrint < timeInterval) return;
    lastTimePrint = now;

    bool casOK = false;
    if (WiFi.status() == WL_CONNECTED) {
        struct tm timeinfo;
        if (getLocalTime(&timeinfo)) {
            hodina = timeinfo.tm_hour;
            minuta = timeinfo.tm_min;
            hodina1 = hodina / 10;
            hodina2 = hodina % 10;
            minuta1 = minuta / 10;
            minuta2 = minuta % 10;
            DateTime nowRTC(1900 + timeinfo.tm_year, 1 + timeinfo.tm_mon,
                           timeinfo.tm_mday, timeinfo.tm_hour,
                           timeinfo.tm_min, timeinfo.tm_sec);
            rtc.adjust(nowRTC);
            casOK = true;
        }
    }
    if (!casOK) {
        DateTime nowRTC = rtc.now();
    }
}

```

```

    hodina = nowRTC.hour();
    minuta = nowRTC.minute();
    hodina1 = hodina / 10;
    hodina2 = hodina % 10;
    minuta1 = minuta / 10;
    minuta2 = minuta % 10;
}

FastLED.clear();

EVERY_N_MILLISECONDS(1000) {
    vykreslicislo(0, 6, hodina1, R, G, B);
    vykreslicislo(4, 6, hodina2, R, G, B);
    vykreslicislo(8, 6, minuta1, R, G, B);
    vykreslicislo(12, 6, minuta2, R, G, B);

    if (dvojtecka) {
        leds[XY(7, 3)] = CHSV(0, 0, 128);
        leds[XY(7, 5)] = CHSV(0, 0, 128);
        dvojtecka = false;
    } else {
        leds[XY(7, 3)] = CRGB::Black;
        leds[XY(7, 5)] = CRGB::Black;
        dvojtecka = true;
    }

    vlhkost = dht.readHumidity();
    teplota = dht.readTemperature();
    if (!isnan(vlhkost) && !isnan(teplota)) {
        vlhkost1 = vlhkost / 10;
        vlhkost2 = vlhkost % 10;
        teplota1 = teplota / 10;
        teplota2 = teplota % 10;
    }

    if (pocetsekundDHT < 5) {
        for (int i = 128; i <= 255; i++) leds[i] = CRGB::Black;
        vykreslicislo(0, 13, teplota1, R, G, B);
        vykreslicislo(4, 13, teplota2, R, G, B);
        leds[XY(8, 9)] = CRGB(R, G, B);
        leds[XY(10, 9)] = CRGB(R, G, B);
        leds[XY(11, 9)] = CRGB(R, G, B);
        leds[XY(10, 10)] = CRGB(R, G, B);
        leds[XY(10, 11)] = CRGB(R, G, B);
        leds[XY(11, 11)] = CRGB(R, G, B);
    }
    if (pocetsekundDHT >= 5 && pocetsekundDHT < 10) {
        for (int i = 128; i <= 255; i++) leds[i] = CRGB::Black;
        vykreslicislo(0, 13, vlhkost1, R, G, B);
    }
}

```

```

    vykreslicislo(4, 13, vlhkost2, R, G, B);
    leds[XY(8, 13)] = CRGB(R, G, B);
    leds[XY(9, 12)] = CRGB(R, G, B);
    leds[XY(10, 11)] = CRGB(R, G, B);
    leds[XY(11, 10)] = CRGB(R, G, B);
    leds[XY(12, 9)] = CRGB(R, G, B);
    leds[XY(8, 10)] = CRGB(R, G, B);
    leds[XY(8, 9)] = CRGB(R, G, B);
    leds[XY(9, 9)] = CRGB(R, G, B);
    leds[XY(12, 12)] = CRGB(R, G, B);
    leds[XY(12, 13)] = CRGB(R, G, B);
    leds[XY(11, 13)] = CRGB(R, G, B);
    leds[XY(9, 10)] = CRGB(R, G, B);
    leds[XY(11, 12)] = CRGB(R, G, B);
}

pocetsekundDHT++;
if (pocetsekundDHT >= 10) pocetsekundDHT = 0;

    leds[XY(0, 0)] = (WiFi.status() == WL_CONNECTED) ? CRGB(0, 255, 0) :
CRGB(255, 0, 0);

    FastLED.show();
}
}

```

5.2 Clock_wifi.h

```

#pragma once
#include <Arduino.h>

void setRTCfromNTP();
void manageWiFiNonBlocking();
void nacticaszwifi();

```